INSIDE ZUSTAND

State Management for React

Stores, selectors, middleware, and persistence with the pmndrs Zustand library

Covers Zustand 5.0.16 (tag v5.0.16, commit 382d29c)

An unofficial, independently written guide. Not affiliated with or endorsed by pmndrs or the Zustand maintainers.

Contents

	Preface	**6**
	Who this book is for	6
	What this book covers	7
	Supported versions and environments	7
	How to read this book	8
	Conventions	8
Chapter 1	**Introduction to Zustand**	**10**
	Roadmap	10
	The problem Zustand solves	10
	What Zustand does not do	11
	Where Zustand fits	11
	Core vocabulary	12
	The parts of Zustand	13
	A first store	14
Chapter 2	**Stores and State Updates**	**16**
	Roadmap	16
	What a store is	16
	How a store is created	17
	How setState computes the next state	17
	How listeners are notified	19
	Immutability rules	20
	Actions and where logic lives	21
	Store invariants	21
Chapter 3	**Selectors and Rendering in React**	**23**
	Roadmap	23
	How a component subscribes to a store	24
	When a component re-renders	25
	Selectors must return stable values	25
	Shallow comparison with `useShallow`	26
	Custom equality with `zustand/traditional`	27
	Server rendering and the initial state	27
	Module stores and scoped stores	28
Chapter 4	**Middleware and Store Types**	**29**

Roadmap 29
What middleware is 29
How middleware composes 30
Why `create` is curried in TypeScript 31
How middleware changes the store type 32

Chapter 5 **Building Stores** **33**
Roadmap 33
Define a store with state and actions 34
Update nested state 37
Run asynchronous actions 37
Read and write state outside components 38
React to changes without rendering 39
Reset a store 39
Split a large store into slices 40
Troubleshooting 41

Chapter 6 **Using Stores in Components** **42**
Roadmap 42
Read a value in a component 42
Select several values 43
Compute derived values 44
Use a vanilla store in React 45
Scope a store to a component tree 45
Create a store per server request 46
Troubleshooting 47

Chapter 7 **Persisting State** **49**
Roadmap 49
About persistence 49
The hydration lifecycle 50
Persist a store 53
Version and migrate persisted state 54
Merge nested persisted state 56
Wait for hydration with asynchronous storage 57
Control hydration for server rendering 59
Store values JSON cannot represent 60
Clear persisted state 61
Troubleshooting 61
Persist reference 62

Chapter 8 **Using Middleware** 64
Roadmap 64
Compose a middleware stack 64
Inspect a store with Redux DevTools 66
Write updates as mutations with Immer 66
Subscribe to part of the state 66
Infer the state type with combine 67
Write your own middleware 67
Troubleshooting 70

Chapter 9 **Testing Stores** 71
Roadmap 71
Test store logic without React 72
Reset module stores between tests 72
Test components with an injected store 73
Test persisted stores 74
Troubleshooting 75

Chapter 10 **Zustand Reference** 77
Roadmap 77
Entry points 78
Shared rules 78
`createStore()` 79
`StoreApi` 80
`StateCreator` 80
`create()` 81
`useStore()` 82
`useShallow()` 83
`shallow()` 83
`createWithEqualityFn()` 84
`useStoreWithEqualityFn()` 84
`devtools()` 85
`immer()` 86
`subscribeWithSelector()` 87
`combine()` 87
`redux()` 88

Appendix A **Glossary** 90

Appendix B **Errors and Symptoms** 93

Appendix C **Compatibility and Migration** **94**
Requirements 94
Changes from Zustand 4 94
Experimental and unstable APIs 95

Preface

Zustand is a small state-management library from the pmndrs collective. A Zustand store holds application state outside the React component tree. Components read it through hooks, and plain JavaScript reads and updates it through a handful of methods. This book explains how a store computes and announces each update, how React components subscribe to it, and how the bundled middleware changes that behavior. It then teaches the common tasks and specifies the contract of each public function for lookup.

The book is unofficial. It was written independently of the Zustand project, and the maintainers have not reviewed it.

Who this book is for

This book is for application developers who build React or TypeScript applications with Zustand. You create stores, read them from components and from ordinary modules, add the bundled middleware, and test the result.

The book assumes that you already know:

- modern React: function components, hooks, context, and effects;
- TypeScript at the level of interfaces, union types, generics, and utility types such as `Partial<T>`;
- ES modules and `async` functions.

You do not need to know Redux, Flux, or React's `useSyncExternalStore` hook. Chapter 3 explains the part of `useSyncExternalStore` that decides when a component re-renders.

A smaller group of readers writes its own middleware or types a middleware stack by hand. Sections for those readers are labeled "If you write middleware." Everyone else can skip them without missing anything the other chapters depend on.

What this book covers

The book covers the vanilla store, the React bindings and selectors, module and scoped stores, the six stable bundled middleware, TypeScript typing, custom middleware, and testing. Persistence gets a full chapter, because persisted state has its own lifecycle and failure modes.

The book leaves these topics out:

- Next.js routing and the split between server and client components. Zustand's Next.js guide covers them. The framework-independent pattern appears in Create a store per server request.
- React Native. The package declares a React Native export condition, as Requirements notes, but no listing in this book was run on React Native.
- Jest and Vitest module mocks that reset every store automatically. Zustand's testing guide shows that wiring. The book resets stores without a test runner's mocking features.
- Experimental exports, which Experimental and unstable APIs names without documenting.
- Zustand 3 and 4 APIs, beyond the changes listed in Changes from Zustand 4.
- The Redux DevTools message protocol, the type-level machinery behind middleware types, and third-party libraries built on Zustand.

Supported versions and environments

This book describes Zustand 5.0.16, tag `v5.0.16`, commit `382d29c`. Behavioral statements were checked against the library's source code and test suite at that commit.

Every listing was type-checked with `tsc` 7.0.2 under a strict `tsconfig.json` with `jsx` set to `react-jsx`. Listings that produce output were run with `tsx` 4.23.15 on Node.js 25.9.0, with React 19.3.1, react-dom 19.3.1, and Immer 11.2.9 installed, and the output shown beside a listing is the output that run printed. The test environment has no DOM, so component listings were verified by type-checking and, where a listing renders markup, by rendering it with `renderToString` from `react-dom/server`. The `use-sync-external-store` package was not installed, so listings that import `zustand/traditional` were type-checked only. A statement the book could not verify carries a leading **Unverified:** label.

For the minimum versions of React, TypeScript, and the optional peer packages, and for the differences from Zustand 4, see Compatibility and migration.

How to read this book

Chapters 1 to 4 explain the model, and they build on each other, so read them in order. Chapters 5 to 9 teach tasks. Chapter 5 builds the cart store that Chapters 6 to 9 extend, and after it you can read the task chapters in any order. Chapter 10 and the appendixes are for lookup.

Read this first: Chapter 2, Stores and State Updates, is foundational. Every later chapter applies its rules for merging updates, notifying listeners, and keeping state immutable, so a reader who skips it will find selector, persistence, and middleware behavior hard to predict.

Use this table to choose a path:

If you want to	Read	You can skip
Understand the model	Chapters 1 to 4	Chapters 9 and 10
Add a store to a React app	Chapter 1, Chapter 3's sections on re-rendering and stable selectors, Chapters 5 and 6	Chapter 4's typing sections, Chapter 8
Persist state	Chapter 2's update rules, Chapter 7	Chapters 8 and 9
Write middleware	Chapter 4, Write your own middleware in Chapter 8	Chapters 6 and 9
Look up a contract	Chapter 10, Persist reference in Chapter 7, Appendix B	The narrative chapters

Each numbered chapter opens with a roadmap that names its readers, its prerequisites, and the sections you can skip.

Conventions

Listings are written in TypeScript. Each listing shows its imports, the types it uses, and its sample data. A listing that builds on an earlier one imports from a file shown earlier in the same chapter.

Most examples use one sample model, a storefront cart. A cart holds items keyed by SKU, an optional coupon code, and the status of a checkout. Prices are integer cents, so `1250` means $12.50. Integer arithmetic keeps subtotals exact, where sums of fractional dollars would accumulate floating-point rounding error. Define a store with state and actions gives the full model.

Store names follow the kind of object:

- `useCartStore` names a bound hook, the function that `create` returns. The `use` prefix tells React's lint rules to treat it as a hook.
- `cartStore` names a vanilla store, the object that `createStore` returns.
- Selector functions that a module exports start with `select`, as in `selectItemCount`.

When a listing prints output, a `text` block follows the listing and contains exactly what the run printed:

```
import { createStore } from 'zustand/vanilla'

type CartItem = { sku: string; name: string; unitPrice: number; quantity: number }
type CartState = { items: Record<string, CartItem> }

const cartStore = createStore<CartState>()(() => ({
  items: {
    'TEA-01': { sku: 'TEA-01', name: 'Sencha tea', unitPrice: 1250, quantity: 2 },
  },
}))

const tea = cartStore.getState().items['TEA-01']
console.log(tea.name, tea.unitPrice * tea.quantity)
```

```
Sencha tea 2500
```

Callouts have fixed meanings:

Callout	Meaning
Note	Context that helps you understand a behavior but does not change what you do
Important	A condition that a procedure or design needs in order to work correctly
Warning	An action that can lose data, corrupt state, or break rendering, with the safer alternative

"Hydration" by itself means the persist middleware reading stored state into a store; the React meaning is written "React hydration". The Glossary defines the other terms the book uses with a fixed meaning.

CHAPTER 1

Introduction to Zustand

Zustand keeps shared application state in a store, a small object that lives outside the React component tree. Components read a store through a hook, and any other code reads and updates it through the store's methods.

Roadmap

Read this chapter if

You are new to Zustand or are deciding whether it fits your application.

Before you begin

You need working knowledge of React hooks and TypeScript generics, and a project with `zustand`, `react`, and `tsx` to run the example.

In this chapter you will

- Decide whether Zustand suits your shared state.
- Create a store, read it in a component, and update it from a script.

You can skip

If you already use Zustand 5, skim Core vocabulary for the book's fixed terms, such as "slice," and continue with Chapter 2.

The problem Zustand solves

Some state belongs to no single component. A shopping cart is read by a header badge and a checkout form, and it is changed by code outside React, such as a WebSocket handler or a test.

Lifting that state into a common parent means passing it through components that do not use it. A React context avoids the passing, but every reader re-renders when the value changes. Neither option lets code outside React reach the state, because hooks run only inside components.

Zustand moves the state into a store outside the tree. A component subscribes with a selector and re-renders when the part it selected changes; When a component re-renders gives the exact rule.

What Zustand does not do

Zustand does not	Consequence
Cache server data	Use a server-cache library such as TanStack Query alongside the store.
Enforce immutability	An in-place mutation goes unnoticed; follow Immutability rules.
Memoize derived state	A derived array or object needs the handling in Compute derived values.
Provide async primitives	Loading and error states are fields you define, as in Run asynchronous actions.
Require or supply a provider	A module store needs none; for one store per subtree, see Scope a store to a component tree.

Where Zustand fits

Approach	Where state lives	What re-renders a reader	Setup
`useState`	One component	That component	None
React context	A provider	Every reader, on any value change	A context and a provider
Redux Toolkit	A store, reached through a provider	A reader whose `useSelector` result changes	A store, slice reducers, and a provider
Server-cache library, such as TanStack Query	A query cache, keyed by query	A reader whose query result changes	A client, a provider, and a fetch function per query
Zustand	Stores outside the tree, with no provider	A reader whose selected value changes	One `create` call

Zustand's comparison page covers other client-state libraries.

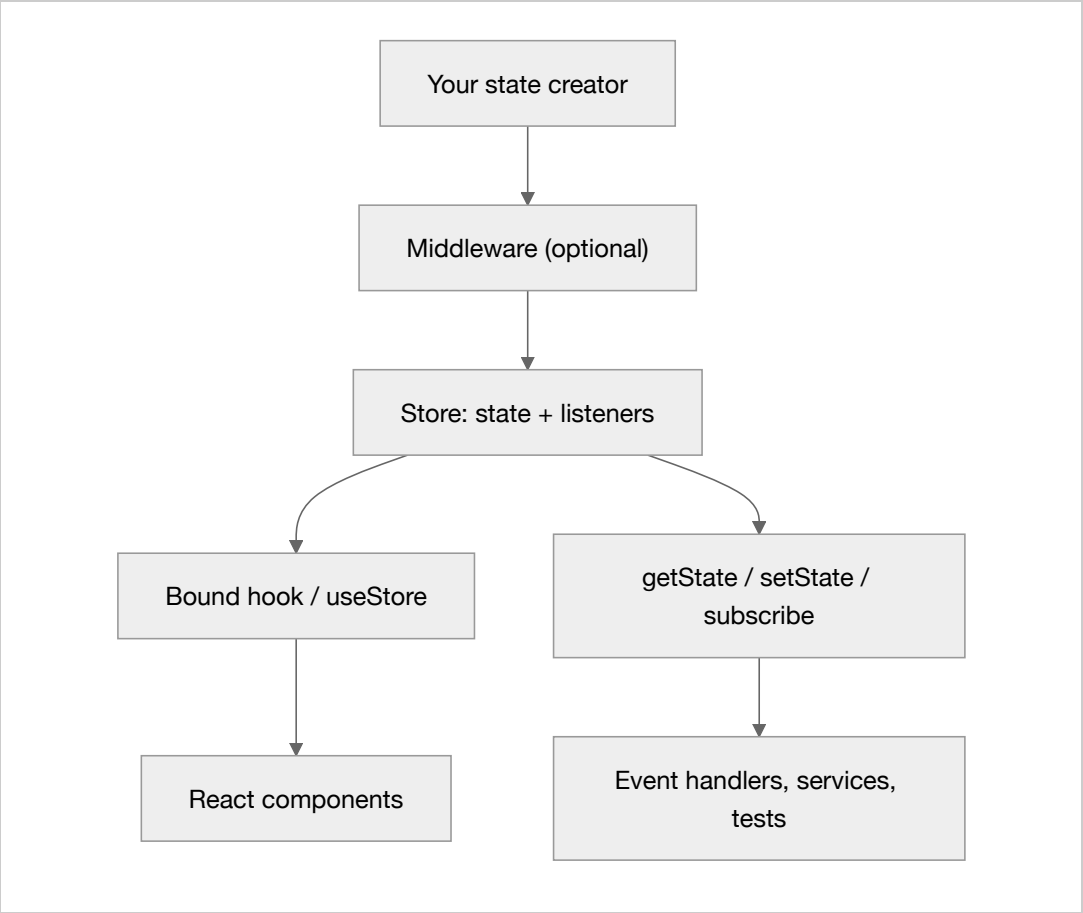
Core vocabulary

Each term links to the section that explains it; the Glossary defines the rest.

- **Store**: an object outside React that holds one current state and notifies its subscribers when that state changes.
- **State**: the value a store holds, usually an object of data fields and actions.
- **Action**: a function stored in the state, such as `addItem`, that updates the store when called.
- **State creator**: the function you pass to `create`, `createStore`, or a middleware; its return value is the initial state.
- **Selector**: a function from the state to the part one reader uses.
- **Selected value**: what a selector returns. Zustand's type parameters call it a "slice"; this book does not.
- **Slice**: in this book, only a state creator that builds part of a store's state.
- **Bound hook**: the function `create` returns, a hook with the store's methods attached.
- **Vanilla store**: the store object `createStore` returns, with no hook.
- **Middleware**: a function that takes a state creator and returns one.

The parts of Zustand

Figure 1-1: Components and plain code reach the same store, through a hook or through the store API.



Because the state is kept only in the store, a component and a WebSocket handler read the same value. The store itself comes from `zustand/vanilla`, which does not depend on React; Entry points lists the rest.

A first store

Install the package with `npm install zustand`. This store keeps the quantity of each item in a cart, keyed by SKU, and an `addItem` action.

```
// cart-store.ts
import { create } from 'zustand'

export type CartState = {
  items: Record<string, number> // quantity by sku
  addItem: (sku: string, quantity?: number) => void
}

export const useCartStore = create<CartState>()((set) => ({
  items: {},
  addItem: (sku, quantity = 1) =>
    set((state) => ({
      items: { ...state.items, [sku]: (state.items[sku] ?? 0) + quantity },
    })),
}))
```

The empty first call in `create<CartState>()(…)` lets you name the state type while TypeScript infers the rest; Why `create` is curried in TypeScript explains why.

A component calls the bound hook with a selector, and needs no provider:

```
// CartBadge.tsx
import { useCartStore } from './cart-store'

export function CartBadge() {
  const count = useCartStore((state) =>
    Object.values(state.items).reduce((n, quantity) => n + quantity, 0),
  )
  return <span aria-label="Items in cart">{count}</span>
}
```

Code outside React uses the methods attached to the hook:

```
// add-items.ts
import { useCartStore } from './cart-store'

const { addItem } = useCartStore.getState()
addItem('TEA-01')
addItem('TEA-01')
```

```
addItem('MUG-02', 2)

console.log(useCartStore.getState().items)
```

Run it with `npx tsx add-items.ts`. It prints:

```
{ 'TEA-01': 2, 'MUG-02': 2 }
```

Each call updates the store before it returns. Chapter 2 explains how a store computes and announces every update.

CHAPTER 2

Stores and State Updates

Read this first. This chapter explains the store: how it holds state, computes each update, and notifies subscribers. Selectors, middleware, and persistence all act on these updates, so every later chapter applies these rules.

Roadmap

Read this chapter if

You use Zustand in any form.

Before you begin

Read Core vocabulary. The listings run in Node with `zustand/vanilla`.

In this chapter you will

- predict the next state and the notifications for any `setState` call;
- learn the immutability rules that make change detection work;
- see where actions and asynchronous logic belong.

You can skip

Nothing, if you are new to Zustand. If you know Zustand 4 well, read only How listeners are notified and Immutability rules.

What a store is

A store is a closure that holds one current state value and a set of listener functions. Four methods, specified in `StoreApi`, are the only way to reach the state: `getState`, `setState`, `subscribe`, and `getInitialState`. The store does not depend on React, and `create` builds

the same kind of store with these methods attached to the hook it returns.

A store lives as long as something references it, so a module store lives for the life of the process, or until the page unloads. Zustand has no method that disposes a store, and a listener keeps alive everything it closes over until you call the function that `subscribe` returned.

How a store is created

`createStore` and `create` take a *state creator*, a function that returns the initial state. The store calls it once, synchronously, before `createStore` returns, passing `set` (the store's `setState`), `get` (its `getState`), and the store API object. The return value becomes the current state, and `getInitialState()` returns that same value for the rest of the store's life.

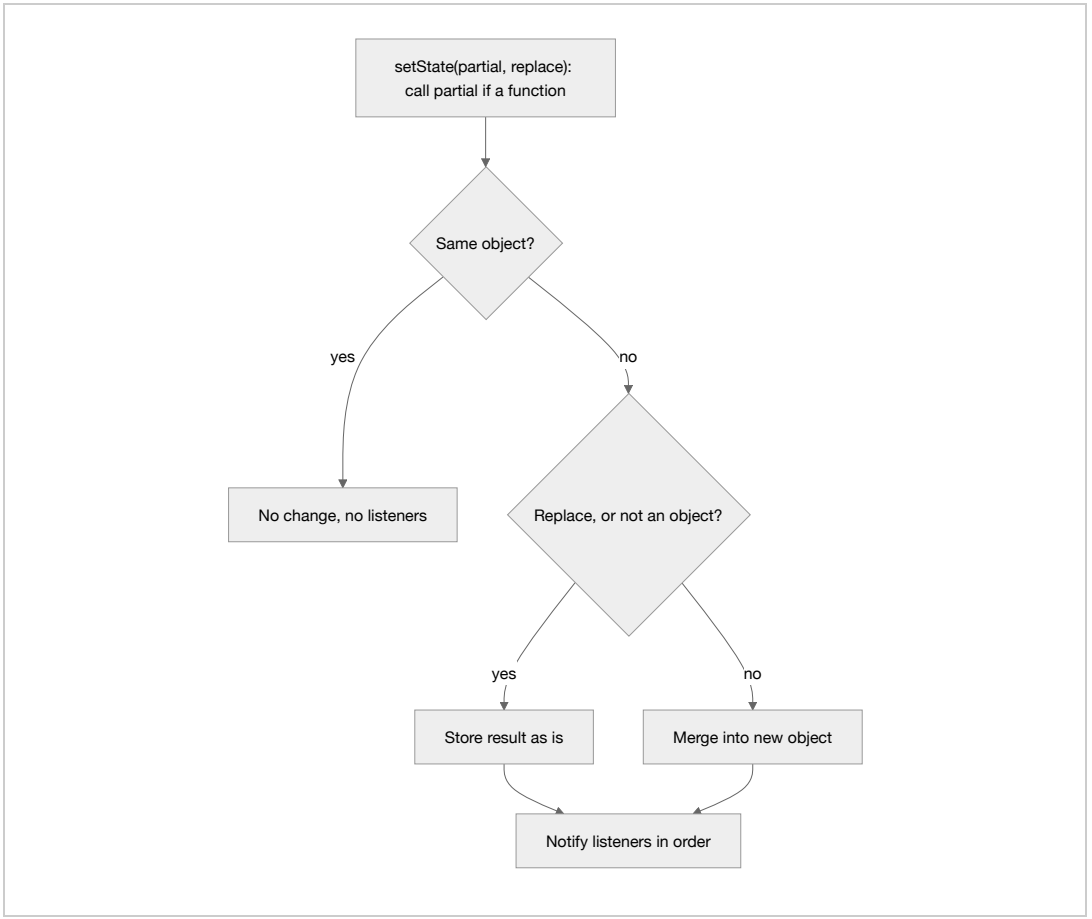
Actions are ordinary state fields whose values are functions. An action runs only after the state creator has returned, so `get()` inside it sees the current state. Called directly in the creator's body, before the state exists, `get()` returns `undefined`.

How setState computes the next state

`setState` accepts either a value or an updater function, and computes the next state in these steps:

1. If the argument is a function, `setState` calls it with the current state and uses its return value.
2. If that result is identical to the current state under `Object.is`, `setState` returns without storing anything or calling any listener.
3. If `replace` is `true`, or `replace` is omitted and the result is not an object (or is `null`), the store keeps the result as is. An explicit `replace` of `false` always merges.
4. Otherwise the store keeps a new object built with `Object.assign({}, state, result)`.
5. The store calls every listener.

Figure 2-1: setState merges into a new object unless the result is identical or replacement is requested.



The second decision tests the result's type only when you omit `replace`. Passing `false` merges even a primitive, so `setState(5, false)` on a store whose state is a number stores `{}`.

For an object state, the only way to skip notification is to return the current state object itself. Any other object produces a new state object, so it notifies even when every value is unchanged.

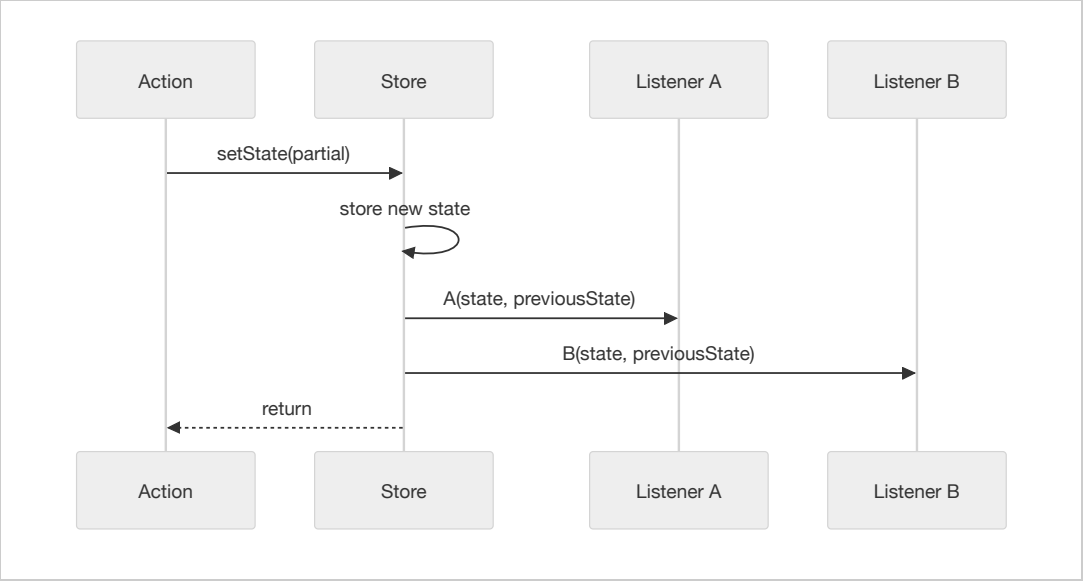
The merge is one level deep: each top-level field in the result replaces its namesake, fields you leave out keep their values, and a nested object replaces the old one whole.

With `replace: true`, the `setState` type accepts only a complete state, while a merge accepts `Partial<T>`. TypeScript checks this because the store does not: at run time an incomplete object becomes the whole state, and the omitted fields are gone.

How listeners are notified

The store calls listeners synchronously inside `setState`, after it has stored the new state and before `setState` returns. It calls them in subscription order and passes each one `(state, previousState)`. Zustand does not batch notifications, so listener cost adds to every update.

Figure 2-2: Listeners run synchronously, in subscription order, before setState returns.



Code that runs after `setState` returns therefore sees the new state and knows that every listener has run.

The store does not catch errors. If a listener throws, the error propagates out of `setState` after the new state is stored, and later listeners, including React subscriptions, miss that update. Catch errors inside a listener when the others must still run.

A listener can call `setState`. The nested call notifies every listener before the outer loop continues, so a later listener receives the nested change before the outer one. Consider a store with listeners A and B, subscribed in that order:

1. `count` is `0`, and an action sets it to `1`.
2. Listener A receives `0 -> 1` and calls `setState` to set `count` to `2`.
3. That nested call notifies A and then B, so B receives `1 -> 2`.
4. The outer loop resumes and calls B with the outer notification, `0 -> 2`. Its `previousState` is stale, because `count` was `1` in between.

To avoid this, compute both values in the action that makes the first change. One `setState` call then produces one notification, and every listener receives it with an accurate `previous-State`.

Immutability rules

You must create a new object, array, `Map`, or `Set` for every level of the state you change. Zustand and React detect change only by comparing references (When a component re-renders), and Zustand does not freeze your state, so an in-place mutation goes unnoticed in one of two ways:

- If the updater mutates the state and returns the same object, `Object.is` reports no change, so no listener runs.
- If the updater mutates a nested object and returns a merge, the store notifies, but `previousState` already contains the mutation.

```
import { createStore } from 'zustand/vanilla'

type CartState = { items: Record<string, { quantity: number }> }

const cartStore = createStore<CartState>()(() => ({
  items: { 'TEA-01': { quantity: 1 } },
}))

cartStore.subscribe((state, prev) =>
  console.log(prev.items['TEA-01'].quantity, '->', state.items['TEA-01'].quantity),
)

// Wrong: changes the existing item object in place.
cartStore.setState((state) => {
  state.items['TEA-01'].quantity = 2
  return { items: state.items }
})

// Right: new item, new items record, new state object.
cartStore.setState((state) => ({
```

```
items: { ...state.items, 'TEA-01': { ...state.items['TEA-01'], quantity: 3 } },
}))
```

```
2 -> 2
2 -> 3
```

The wrong update changed an object that `previousState` shares. The right update copies only the path to the change, so other items keep their references. To write the same update as a mutation, use Immer, which produces the new references for you.

Actions and where logic lives

An action is defined in the state creator, so it closes over `set` and `get`. Inside an action, read state with `get()` when you need it, because a value captured earlier can be stale.

This matters most in asynchronous actions, which are ordinary `async` functions that call `set` when they start and again when results arrive; Zustand has no asynchronous concept. If two concurrent calls each build their update from state captured before an `await`, the second write discards the first. Run asynchronous actions shows the complete pattern.

Actions survive every merge, because a merge keeps the fields the partial state leaves out. A replacement keeps only what it contains, so `setState(value, true)` removes every action that `value` does not include. Replacing with `getInitialState()` keeps the actions, which is how Reset a store works.

Module-level functions that call `store.setState` also work as actions, bound to one store instance; see Read and write state outside components.

Store invariants

These facts hold for the core store; middleware can add behavior around them.

- `getState()` returns the latest state synchronously.
- Zustand never mutates a state object.
- `getInitialState()` never changes.
- Listeners run only from `setState`, and only when the next state is not `Object.is`-identical to the current one.

- The core store throws no errors of its own; an error from `setState` comes from your updater or a listener.

CHAPTER 3

Selectors and Rendering in React

This chapter explains how React components read a Zustand store and which updates make them render. Chapter 6 turns its rules into component tasks.

Roadmap

Read this chapter if

You read a store from React components, or a component renders more or less often than you expect.

Before you begin

Read How setState computes the next state and How listeners are notified.

In this chapter you will

- Predict from a selector which updates re-render a component.
- Recognize unstable selectors and choose a fix.
- Choose between a module store and a scoped store.

You can skip

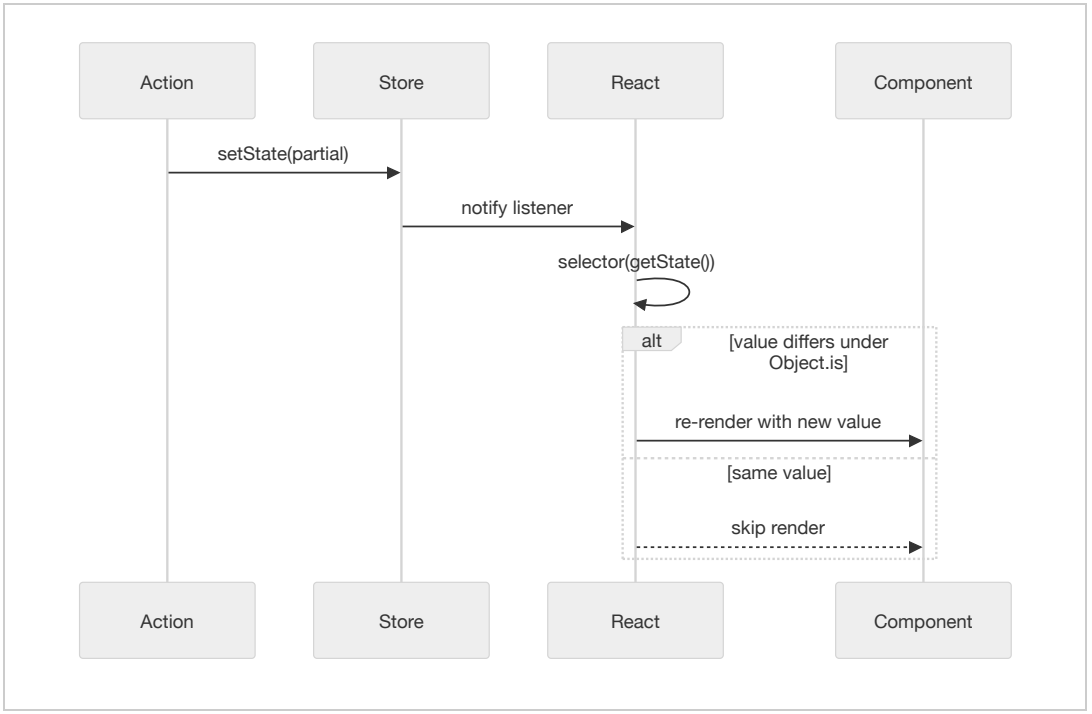
Skip Custom equality with `zustand/traditional` unless you port Zustand 4 code. Skim the server rendering section if your application renders only in the browser.

How a component subscribes to a store

`useStore(store, selector)` connects a component to a store through React's `useSyncExternalStore` hook. It passes React three functions: the store's `subscribe` method, a snapshot function that returns `selector(store.getState())`, and a server snapshot function that returns `selector(store.getInitialState())`. The snapshot's result is the **selected value**, the value the component receives.

A bound hook, the function `create` returns, is `useStore` with its store already applied, plus the store's methods, so plain code can call `useCartStore.getState()`. No provider is involved: each hook call subscribes directly to the store object. The contracts are in `useStore()` and `create()`.

Figure 3-1: A component re-renders only when its selected value changes under Object.is.



Every subscribed component runs its selector after every notification, whether or not it then renders, so keep selectors cheap and free of side effects (Shared rules).

When a component re-renders

After each notification, React calls the snapshot function and compares the new selected value with the value the component last rendered, using `Object.is`. React renders the component again only if the two differ. The store notifies on every change, so the selector is what filters notifications for one component. It does not prevent ordinary React renders, such as when the parent renders.

`Object.is` compares objects, arrays, and functions by reference, so the selector's shape decides how often a component renders:

Selector returns	Example	The component re-renders
The whole state	`useCartStore()`	After every notification, because each change creates a new state object
A primitive	`(s) => s.coupon`	When that value changes
An object stored in state	`(s) => s.items`	When an update stores a new reference there
An action	`(s) => s.addItem`	Not on updates, because merges keep the function the state creator made
A newly built object or array	`(s) => ({ coupon: s.coupon })`	Again on every check, until React throws "Maximum update depth exceeded" (next section)

An update that mutates an object in place keeps its reference, so components that select that object do not re-render (Immutability rules).

Selectors must return stable values

A selector is stable when two calls on the same state return values that are equal under `Object.is`. A selector that builds a new object, array, or function on each call is unstable, including `Object.values(s.items)` and fallbacks such as `s.lastOrderSkus ?? []`.

React requires the snapshot to stay the same while the store is unchanged. With an unstable selector, React sees a new snapshot each time it checks, renders again, and gets another new value. In development React logs "The result of getSnapshot should be cached to avoid an infinite loop" and then throws "Maximum update depth exceeded". A selector that always

builds a new value fails at mount, before any update. A fallback such as `?? []` builds a new value only when the field is missing, so the component fails only for the props or state that reach that branch.

Situation	Fix
The component needs several fields	Call the hook once per field
The selector must build an object or array from state	Wrap it in `useShallow`: `useCartStore(useShallow((s) => Object.values(s.items)))`
A missing field needs an object or array default	Return a constant created once at module level, such as `s.lastOrderSkus ?? NO_SKUS`
The value is computed, such as a sorted list	Select the stored input and compute during render (Compute derived values)
You need your own equality rule	Use `zustand/traditional` (Custom equality with `zustand/traditional`)

This requirement is new in Zustand 5, so code written for Zustand 4 can break on it (Changes from Zustand 4).

Shallow comparison with `useShallow`

`useShallow(selector)` returns a selector that remembers its previous result. When a new result is shallowly equal to the remembered one, it returns the remembered reference, so `useStore` sees an unchanged value until an element changes.

Shallow equality looks one level deep. Two values are shallowly equal when their top-level elements or properties are equal under `Object.is`, so nested objects are equal only when they are the same reference. `shallow()` gives the full comparison rules.

First value	Second value	`shallow`
`{ a: 1, b: 2 }`	`{ b: 2, a: 1 }`	`true`
`{ item: { quantity: 2 } }`	`{ item: { quantity: 2 } }`	`false`
`['TEA-01', 'MUG-02']`	`['MUG-02', 'TEA-01']`	`false`

The hook's contract is in `useShallow()`.

Custom equality with `zustand/traditional`

`zustand/traditional` provides counterparts of `create` and `useStore` that accept an **equality function**, which receives the previous and next selected values and returns `true` to skip the render. They need the optional `use-sync-external-store` package (Requirements). Choose them when you port Zustand 4 code that passes `shallow` as the hook's second argument, or when you need a rule that `shallow` cannot express, such as "the same SKUs in any order". Otherwise prefer `useShallow`, which needs no extra package. The contracts are in `createWithEqualityFn()` and `useStoreWithEqualityFn()`.

Server rendering and the initial state

During server rendering, and in the first render of React hydration in the browser, React reads the server snapshot, `selector(getInitialState())`, not `getState()`. Server markup therefore shows the state the state creator returned, even when server code updated a module store before rendering, as the following listing shows.

```
import { renderToString } from 'react-dom/server'
import { create } from 'zustand'

const useCartStore = create<{ coupon: string | null }>()(() => ({ coupon: null }))

function CouponBanner() {
  const coupon = useCartStore((s) => s.coupon)
  return <p>{coupon ?? 'No coupon'}</p>
}

useCartStore.setState({ coupon: 'SPRING10' })
console.log('getState():', useCartStore.getState().coupon)
console.log('server markup:', renderToString(<CouponBanner />))
```

The script prints:

```
getState(): SPRING10
server markup: <p>No coupon</p>
```

To render request-specific state on the server, make it the initial state of a store created for that request. Persisted stores add their own step (Control hydration for server rendering).

> **WARNING**
> Do not write request or user data into a module store on a server. Every request in the process shares the store, so one request can read data that an overlapping request wrote. Create a store per request instead (Create a store per server request).

Module stores and scoped stores

A **module store** is created at the top level of a module. A **scoped store** is created by a provider component and passed to descendants through React context.

	Module store	Scoped store
Created by	`create` at module level	`createStore` in a provider, once per mount
Instances	One per browser page or server process	One per provider: per subtree, request, or test
Initial state	Fixed by the state creator	Can come from props
Access	Import the hook, also outside React	Through context, inside the provider only

Use a module store when the application renders only in the browser and needs one instance of the state. Use a scoped store when you render request data on the server, need several independent instances, or take initial state from props. Module stores in tests need a reset between tests (Reset module stores between tests). Scope a store to a component tree builds the provider, and Create a store per server request uses it on the server.

CHAPTER 4

Middleware and Store Types

This chapter explains how middleware changes a store and how TypeScript types the result. The middleware tasks in Chapter 8 rely on it.

Roadmap

Read this chapter if

You use Zustand's middleware, write stores in TypeScript, or plan to write a middleware.

Before you begin

Read How a store is created and How listeners are notified.

In this chapter you will

- Learn what a middleware can change and how a stack orders an update.
- Learn why TypeScript code calls `create<T>()(…)`.

You can skip

Skip the last two sections if you write JavaScript.

What middleware is

A middleware is a function that takes a state creator and returns a new state creator. You apply it by wrapping the creator you pass to `createStore` or `create`, as in `create<T>()(devtools(creator))`, and it behaves the same with either.

The middleware does its work when the store runs the creator, once, during creation. It receives `set`, `get`, and the store API, calls the creator it wraps, and can:

- wrap `set` before passing it inward, so that your actions' updates pass through it;
- replace store API methods such as `setState` or `subscribe`, so that outside calls pass through it too;
- add members to the store API, such as `store.persist`;
- transform the state the inner creator returns, which becomes the initial state.

Zustand bundles six middleware:

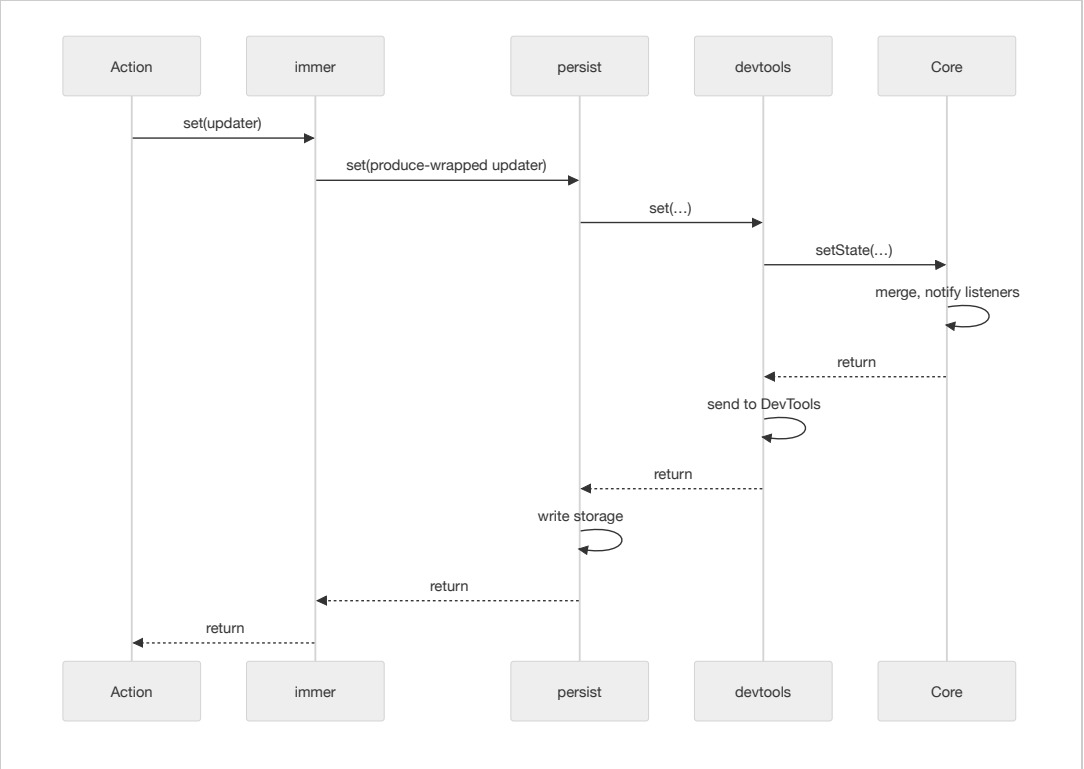
Middleware	Purpose	Guide
`devtools`	Records updates in the Redux DevTools extension	Inspect a store with Redux DevTools
`persist`	Saves state to storage and restores it	About persistence
`immer`	Lets an update mutate a draft	Write updates as mutations with Immer
`subscribeWithSelector`	Calls a listener only when a selected value changes	Subscribe to part of the state
`combine`	Infers the state type from an initial object	Infer the state type with combine
`redux`	Updates state with a reducer and actions	`redux()`

How middleware composes

A stack nests middleware calls, as in `devtools(persist(immer(creator)))`. The store runs the outermost state creator, so `devtools` receives the core `set` first, and each layer passes its own wrapped `set` inward.

An action's `set` call therefore runs through the layers innermost first and reaches the core `setState` last. A layer can act before forwarding the call, as `immer` does when it wraps a draft updater in Immer's `produce`, or after the call returns, when the core has stored the new state and every listener has run. On the way out, `devtools` sends its message before `persist` writes storage, because the layer nearer the core regains control first. Each layer also installs its wrapper as `store.setState`, so calls from outside the store take the same path.

Figure 4-1: An action's set call passes through each middleware, innermost first, and the layer nearest the core finishes its work first on the way back.



Put `devtools` outside every other middleware, because it records only updates that pass through its wrapper. `persist`, for example, hydrates the store through the `set` it received, so in `persist(devtools(creator))` the DevTools show the creator's defaults while the store holds the restored state. Compose a middleware stack builds the cart stack in this order.

Why `create` is curried in TypeScript

`create` and `createStore` have two type parameters: the state type `T`, and the list of mutators the middleware stack applies. The creator's `set` and `get` parameters get their types from `T`, so the creator itself gives TypeScript nothing to infer `T` from. You write `T` yourself. TypeScript also cannot infer some type arguments of a call while you supply others. Once you write `T`,

the mutator list takes its default, the empty list, which a middleware's result does not match. Zustand therefore splits the call: `create<T>()` takes only `T` and returns a function that infers the mutator list from your creator.

Without middleware, the one-call form `create<T>(creator)` also type-checks. `create(combine(…))` needs no type argument, because `combine` infers the state type.

How middleware changes the store type

A middleware that changes the store API declares a mutator: a type-level transformation of the store, registered in Zustand's `StoreMutators` interface under an identifier and written as a pair such as `['zustand/immer', never]`. Each middleware's return type puts its mutator in front of those of the creator it wraps, so a stack yields one list. `create` applies that list to `StoreApi<T>`, which is why `store.persist` exists in the type only when `persist` is in the stack.

The same lists appear in `StateCreator<T, Mis, Mos, U>`. `T` is the full state type. `Mis` lists the mutators applied outside this creator, which type the `set`, `get`, and API it receives. `Mos` lists the mutators this creator applies, and `U` is the type it returns. You write these parameters in two cases: a slice under middleware passes the stack's list as `Mis` (Split a large store into slices), and a middleware you write types both the creator it accepts and the one it returns (Write your own middleware).

Write middleware calls inline, inside `create<T>()(…)`, so that each one infers `T` and the outer mutators from the enclosing call. A middleware call assigned to a variable first has nothing to infer from, so its updater parameters become implicit `any`. For a single middleware built separately, write its type argument, as in `devtools<CartState>(creator)`. That argument leaves the middleware's other type parameters at their defaults, so it fails for a stack, and for `persist` with `partialize`, whose persisted type then defaults to `T`. Write a stack inline, or type its innermost creator in full with `StateCreator<CartState, Mis, Mos>`, listing every mutator in the stack as `Mis`.

CHAPTER 5

Building Stores

This chapter builds the storefront cart store that the rest of the book uses: its state and actions, an asynchronous checkout, subscriptions outside React, resets, and slices. Chapters 6 to 9 extend the same store with components, persistence, middleware, and tests.

Roadmap

Read this chapter if

You are adding a store, or want the cart model that Chapters 6 to 9 extend.

Before you begin

Read How setState computes the next state and Actions and where logic lives. Run scripts with `npx tsx <file>.ts` as ES modules.

In this chapter you will

- define the cart store as a vanilla store factory and as a module store;
- write synchronous and asynchronous actions, and run checkout against a fake backend;
- read, subscribe to, and reset a store outside components;
- split a store into slices.

You can skip

Split a large store into slices, if one state creator fits your store.

Define a store with state and actions

The cart keys its line items by SKU, so that an update to one line touches one key. Prices are integer cents. Create `cart.ts`:

```
import { create, createStore, type StateCreator } from 'zustand'

export type CartItem = {
  sku: string
  name: string
  unitPrice: number // integer cents
  quantity: number
}

export type CheckoutState = {
  status: 'idle' | 'submitting' | 'failed' | 'done'
  orderId: string | null
  error: string | null
}

export type CartState = {
  items: Record<string, CartItem> // keyed by sku
  coupon: string | null
  checkout: CheckoutState
  addItem: (item: Omit<CartItem, 'quantity'>, quantity?: number) => void
  setQuantity: (sku: string, quantity: number) => void // quantity < 1 removes
  removeItem: (sku: string) => void
  applyCoupon: (code: string | null) => void
  submitOrder: (api: OrdersApi) => Promise<void>
  reset: () => void
}

export class OrderRejectedError extends Error {}
export type OrdersApi = {
  submit: (lines: CartItem[], coupon: string | null) => Promise<{ orderId: string }>
}

export const selectItemCount = (s: CartState) =>
  Object.values(s.items).reduce((n, i) => n + i.quantity, 0)
export const selectSubtotal = (s: CartState) =>
  Object.values(s.items).reduce((c, i) => c + i.unitPrice * i.quantity, 0)

export const TEA = { sku: 'TEA-01', name: 'Sencha tea', unitPrice: 1250 }
export const MUG = { sku: 'MUG-02', name: 'Stoneware mug', unitPrice: 1800 }
```

```
const cartStateCreator =
  (initialItems: Record<string, CartItem> = {}): StateCreator<CartState> =>
  (set, get, store) => ({
    items: initialItems,
    coupon: null,
    checkout: idleCheckout,

    addItem: (item, quantity = 1) =>
      set((state) => {
        const current = state.items[item.sku]
        const next = (current?.quantity ?? 0) + quantity
        return { items: { ...state.items, [item.sku]: { ...item, quantity: next } }
}
      }),

    setQuantity: (sku, quantity) => {
      if (quantity < 1) {
        get().removeItem(sku)
        return
      }
      set((state) => {
        const item = state.items[sku]
        if (!item) return state
        return { items: { ...state.items, [sku]: { ...item, quantity } } }
      })
    },

    removeItem: (sku) =>
      set((state) => {
        if (!(sku in state.items)) return state
        const { [sku]: _removed, ...items } = state.items
        return { items }
      }),

    applyCoupon: (code) => set({ coupon: code }),

    submitOrder: async (api) => {
      if (get().checkout.status === 'submitting') return
      set({ checkout: { status: 'submitting', orderId: null, error: null } })
      const { items, coupon } = get()
      try {
        const { orderId } = await api.submit(Object.values(items), coupon)
        set({ items: {}, checkout: { status: 'done', orderId, error: null } })
      } catch (error) {
        if (error instanceof OrderRejectedError) {
          set({ checkout: { status: 'failed', orderId: null, error: error.message }
})
          return
        }
```

```
throw error
      }
    },

    reset: () => set(store.getInitialState(), true),
  })

export const createCartStore = (initialItems?: Record<string, CartItem>) =>
  createStore<CartState>()(cartStateCreator(initialItems))

export const useCartStore = create<CartState>()(cartStateCreator())
```

`createCartStore` calls `createStore()` each time, so every provider, server request, or test gets an independent vanilla store whose initial state can include `initialItems`. `create()` builds one module store and returns the bound hook `useCartStore`. Module stores and scoped stores compares them. The selectors compute totals on demand, so a total cannot fall out of step with `items`.

To check the model, create `try-cart.ts`, which adds two teas and two mugs and prints the resulting state:

```
import { createCartStore, MUG, selectItemCount, selectSubtotal, TEA } from './cart'

const cartStore = createCartStore()
cartStore.getState().addItem(TEA)
cartStore.getState().addItem(MUG, 2)
cartStore.getState().addItem(TEA)

const { items, checkout } = cartStore.getState()
for (const item of Object.values(items)) {
  console.log(item.sku, item.quantity, item.unitPrice * item.quantity)
}
console.log(checkout.status)
console.log(selectItemCount(cartStore.getState()),
selectSubtotal(cartStore.getState()))
```

```
TEA-01 2 2500
MUG-02 2 3600
idle
4 6100
```

The second `addItem(TEA)` raised the existing line's quantity, so the record still has one entry per SKU.

Update nested state

`setQuantity` and `removeItem` build a new `items` record and a new object for the changed line, as Immutability rules requires. Unchanged lines keep their identity, so components that select them do not re-render. Both actions return the current `state` for an unknown SKU, which notifies no listener. For deep updates, see Write updates as mutations with Immer.

Run asynchronous actions

`submitOrder` calls `set` before and after it awaits the backend, and `checkout.status` records its progress:

1. If `status` is `submitting`, return. Because `set` is synchronous, a double click's second call sees `submitting`.
2. On success, set `done` with the `orderId` and clear `items`.
3. On `OrderRejectedError`, set `failed`. The cart is unchanged, so a retry submits the same order.
4. On any other error, set `idle` and rethrow, because a stuck `submitting` would block every later call.

A retry after an unexpected error is safe only if the backend did not record the order, so send an idempotency key if your API accepts one.

Create `checkout.ts`, which submits the same cart once against a backend that accepts it and once against one that rejects it. Each run calls `submitOrder` twice without waiting, as a double click would:

```
import { createCartStore, MUG, OrderRejectedError, TEA, type OrdersApi } from
'./cart'

const succeedingApi: OrdersApi = {
  submit: async () => ({ orderId: 'ord_1001' }),
}
const rejectingApi: OrdersApi = {
  submit: async () => {
    throw new OrderRejectedError('MUG-02 is out of stock')
  },
}
```

```
const cartStore = createCartStore()
  cartStore.getState().addItem(TEA)
  cartStore.getState().addItem(MUG)
  cartStore.subscribe((state, previous) => {
    if (state.checkout !== previous.checkout) {
      console.log(label, state.checkout.status)
    }
  })

  const pending = cartStore.getState().submitOrder(api)
  await cartStore.getState().submitOrder(api) // second click: returns at once
  await pending

  const { items, checkout } = cartStore.getState()
  console.log(label, Object.keys(items), checkout.orderId, checkout.error)
}

await run('success:', succeedingApi)
await run('rejected:', rejectingApi)
```

```
success: submitting
success: done
success: [] ord_1001 null
rejected: submitting
rejected: failed
rejected: [ 'TEA-01', 'MUG-02' ] null MUG-02 is out of stock
```

Each run passes through `submitting` once, so the second call submitted nothing. The rejected run keeps both lines, which is what makes its retry safe. Test store logic without React turns the rejection path into a test.

Read and write state outside components

Event handlers outside React, timers, and services call the `StoreApi` methods directly, as in `useCartStore.getState().addItem(TEA)`. Prefer actions to `setState()`, which skips their validation. Module-level functions that call `useCartStore.setState()` also work, but each is bound to one store instance. Inside a component, `getState()` reads without subscribing, so read rendered values through the hook (How a component subscribes to a store).

React to changes without rendering

For a sink outside React, such as a page title, register a listener with `subscribe()`. The listener runs after every update with `(state, previousState)` (How listeners are notified), so compare the part you need yourself. The store keeps a listener until you call the function `subscribe` returns. Create `title.ts`:

```
import { createCartStore, selectItemCount, TEA } from './cart'

const cartStore = createCartStore()
const unsubscribe = cartStore.subscribe((state, previousState) => {
  const count = selectItemCount(state)
  if (count !== selectItemCount(previousState)) {
    console.log(`Cart (${count})`) // a stand-in for document.title
  }
})

cartStore.getState().addItem(TEA)
cartStore.getState().applyCoupon('SPRING') // count unchanged: no output
cartStore.getState().setQuantity('TEA-01', 3)
unsubscribe()
cartStore.getState().removeItem('TEA-01') // no listener: no output
```

```
Cart (1)
Cart (3)
```

The listener ran for the coupon update too, but the count comparison skipped it. In a component, subscribe in an effect and return the unsubscribe function from the effect's cleanup. For subscriptions to a selected value, see Subscribe to part of the state.

Reset a store

`getInitialState()` returns the state the creator built, actions included (How a store is created). To reset, replace the current state with it: `store.setState(store.getInitialState(), true)`, which the cart's `reset` action does. For a partial reset, merge chosen fields from it without the replace flag. A store from `createCartStore(initialItems)` resets to those items, because they were part of its initial state. Under `persist`, also see Clear persisted state.

Split a large store into slices

A slice is a state creator that builds part of a store's state, so separate modules can own features while sharing one `set` and `get`. Type each slice as `StateCreator<StoreState, [], [], Slice>`. The first argument is the whole state, so `get()` inside a slice sees every slice. Create `slices.ts`:

```
import { createStore, type StateCreator } from 'zustand/vanilla'

type CartSlice = { skus: string[]; addItem: (sku: string) => void }
type UiSlice = { isCartOpen: boolean; toggleCart: () => void }
type StoreState = CartSlice & UiSlice

const createCartSlice: StateCreator<StoreState, [], [], CartSlice> = (set, get) =>
({
  skus: [],
  addItem: (sku) => {
    set((s) => ({ skus: [...s.skus, sku] }))
    if (!get().isCartOpen) get().toggleCart() // cross-slice call
  },
})

const createUiSlice: StateCreator<StoreState, [], [], UiSlice> = (set) => ({
  isCartOpen: false,
  toggleCart: () => set((s) => ({ isCartOpen: !s.isCartOpen })),
})

const shopStore = createStore<StoreState>()((...args) => ({
  ...createCartSlice(...args),
  ...createUiSlice(...args),
}))

shopStore.getState().addItem('TEA-01')
console.log(shopStore.getState().isCartOpen) // true
```

Slice fields share one namespace, so a later spread overwrites an earlier field of the same name. Apply middleware once, around the combined creator, because wrapping each slice would create a separate middleware instance per slice. Then pass the stack's mutator list as each slice's second type argument, such as `[['zustand/devtools', never]]`, or a slice under `devtools` cannot name its actions (How middleware changes the store type).

Troubleshooting

Symptom	Cause	Recovery
A component or listener misses an update	An object was changed in place	Build new objects (Immutability rules)
An action is missing after `setState(…, true)`	The replacement omitted the actions	Replace with `getInitialState()`, or drop `true`
Type error on `setState(partial, true)`	Replacement needs a complete state	Drop `true`, or pass a full state
An async action loses concurrent changes	It wrote a value read before the `await`	Read with `get()` after the `await`

CHAPTER 6

Using Stores in Components

This chapter reads the cart store from React components, first through the module store and then through a store scoped to a provider or a server request. It applies the rules from Selectors and Rendering in React.

Roadmap

Read this chapter if

You render store data in React components or need a separate store for each subtree, test, or server request.

Before you begin

Read Selectors must return stable values.

In this chapter you will

1. Select values, actions, and derived values.
2. Read a vanilla store with `useStore`.
3. Scope a store to a component tree or a server request.

You can skip

The last section if you do not render on a server.

Read a value in a component

The listings import `cart.ts` from Define a store with state and actions, with these exports added at its end:

```
// cart.ts (continued)
export type CartStore = ReturnType<typeof createCartStore>

// Sorting builds a new array on every call; read it through useShallow.
export const selectLinesByName = (s: CartState) =>
  Object.values(s.items).sort((a, b) => a.name.localeCompare(b.name))

export const formatCents = (cents: number) => `$${(cents / 100).toFixed(2)}`
```

Call the bound hook with a selector that returns exactly the value the component displays:

```
// cart-line.tsx
import { useCartStore } from './cart'

export function CartLine({ sku }: { sku: string }) {
  const quantity = useCartStore((s) => s.items[sku]?.quantity ?? 0)
  const setQuantity = useCartStore((s) => s.setQuantity)
  return (
    <li>
      {sku}: {quantity}
      <button onClick={() => setQuantity(sku, quantity + 1)}>+</button>
    </li>
  )
}
```

The first selector and its fallback both return a number, so under When a component re-renders the line re-renders only when its own quantity changes. A fallback such as `?? {}` would build a new object on each call. The second selector returns an action, which keeps its reference across merges. Calling `useCartStore()` without a selector returns the whole state and re-renders on every update.

Select several values

A component can call the hook once per value, as `CartLine` does, or call it once with a selector wrapped in `useShallow`:

```
// line-hooks.tsx
import { useShallow } from 'zustand/react/shallow'
import { useCartStore } from './cart'

export function useLine(sku: string) {
  return useCartStore(
```

```
name: s.items[sku]?.name ?? sku,
      quantity: s.items[sku]?.quantity ?? 0,
    })),
  )
}
```

The selector builds a new object on each call, but `useShallow` returns its previous result while the new one is shallowly equal. That comparison is one level deep, so a result that contains a newly built nested object never equals the previous one. Such a result is unstable and fails like any unstable selector.

Separate calls need no wrapper, and each one triggers a re-render only when its own value changes. Use `useShallow` when the component needs an object or array assembled from several fields. It also handles a set of fields that depends on props, which separate calls cannot, because a component must make the same hook calls on every render.

Compute derived values

A selector that computes a number, string, or boolean is stable, so `CartBadge` below re-renders only when the count changes. A selector that builds a new array, for example with `sort`, needs `useShallow`. Export selectors from the store's module so that components, tests, and plain code share them.

```
// summary.tsx
import { useShallow } from 'zustand/react/shallow'
import { formatCents, selectItemCount, selectLinesByName, selectSubtotal,
useCartStore } from './cart'
import { CartLine } from './cart-line'

export function CartBadge() {
  const count = useCartStore(selectItemCount)
  return <span aria-label="Items in cart">{count}</span>
}

export function CartSummary() {
  const subtotal = useCartStore(selectSubtotal)
  const lines = useCartStore(useShallow(selectLinesByName))
  return (
    <section>
      <ul>{lines.map((line) => <CartLine key={line.sku} sku={line.sku} />)}</ul>
      <p>Subtotal: {formatCents(subtotal)}</p>
    </section>
```

```
  )
  }
```

The sorted array holds the stored `CartItem` objects, so `useShallow` keeps the previous array until an item changes. The selector still sorts after every store update. If a derivation is expensive, select `s.items`, which changes only when an item changes, and derive inside `useMemo(() => …, [items])`.

Use a vanilla store in React

Read a store made with `createStore` with `useStore(store, selector)`, which follows the bound hook's rules. `ExtractState` types a wrapper hook:

```
// vanilla.tsx
import { useStore, type ExtractState } from 'zustand'
import { createCartStore } from './cart'

export const cartStore = createCartStore()

export function useCart<U>(selector: (state: ExtractState<typeof cartStore>) => U):
U {
  return useStore(cartStore, selector)
}
```

Scope a store to a component tree

A scoped store is created by a provider and shared with descendants through context, so each mounted provider owns one instance. Module stores and scoped stores explains when to choose one. This provider creates a store from its props:

```
// cart-context.tsx
import { createContext, useContext, useState, type ReactNode } from 'react'
import { useStore } from 'zustand'
import { createCartStore, type CartItem, type CartState, type CartStore } from
'./cart'

const CartStoreContext = createContext<CartStore | null>(null)

export function CartStoreProvider(props: {
  initialItems?: Record<string, CartItem>
  children: ReactNode
```

```
// The initializer runs on the first render only, so each mount owns one store.
  const [store] = useState(() => createCartStore(props.initialItems))
  return <CartStoreContext.Provider value={store}>{props.children}
</CartStoreContext.Provider>
}

export function useCartStoreContext<U>(selector: (state: CartState) => U): U {
  const store = useContext(CartStoreContext)
  if (!store) {
    throw new Error('useCartStoreContext must be called inside <CartStoreProvider>')
  }
  return useStore(store, selector)
}
```

Pass a function to `useState`, because `useState(createCartStore(…))` would build a new store on every render. The context value never changes, so context does not re-render consumers; each component re-renders through its own selector. The provider reads `initialItems` once, so to start over from new items, give it a `key` that changes with them. Those items are the store's initial state, so Reset a store returns to them. Scoped components call the context hook:

```
// scoped-components.tsx
import { selectItemCount } from './cart'
import { useCartStoreContext } from './cart-context'

export function CartBadge() {
  const count = useCartStoreContext(selectItemCount)
  return <span aria-label="Items in cart">{count}</span>
}
```

Create a store per server request

On a server, render the provider once per request. Each render runs its `useState` initializer, so each request gets its own store, discarded with the render:

```
// per-request.tsx
import { renderToString } from 'react-dom/server'
import { TEA } from './cart'
import { CartStoreProvider } from './cart-context'
import { CartBadge } from './scoped-components'

function handleRequest(userId: string) {
  // Stands in for a database read keyed by the signed-in user.
```

```
return renderToString(
    <CartStoreProvider initialItems={items}>
      <CartBadge />
    </CartStoreProvider>,
  )
}

console.log('ana:', handleRequest('ana'))
console.log('ben:', handleRequest('ben'))
```

Running `npx tsx per-request.tsx` prints:

```
ana: <span aria-label="Items in cart">5</span>
ben: <span aria-label="Items in cart">1</span>
```

Each response shows its own items, because server rendering reads `getInitialState()`, which came from that request's props. A module store, by contrast, would share request data across every request in the process (Server rendering and the initial state).

Render the browser's provider with the same `initialItems`, because React hydration reads the client store's initial state. With server components, put the provider in a `'use client'` module. For routing details, follow Zustand's Next.js guide.

Troubleshooting

Symptom	Cause	Recovery
"Maximum update depth exceeded", or a warning that getSnapshot should be cached	A selector builds a new object, array, or function on each call, including fallbacks such as `?? []`, or a new nested object inside a `useShallow` result	Follow Selectors must return stable values
A component re-renders on every store change	No selector, so the component reads the whole state	Select only the fields the component displays
A component does not update	State was mutated in place, or the component reads a different store instance	Follow Immutability rules, and check which store the hook targets
Hydration mismatch in a persisted store	The client's first render reads storage the server cannot	Follow Control hydration for server rendering
`useCartStoreContext must be called inside <CartStoreProvider>`	The component rendered outside the provider	Wrap the tree, or the test's render, in `CartStoreProvider`

CHAPTER 7

Persisting State

The `persist` middleware saves part of a store's state to storage and loads it back when the store is next created.

Roadmap

Read this chapter if

You want state to survive a reload, or you change the stored shape of a persisted store.

Before you begin

Read How middleware composes. The listings run in Node with the Map-backed storage defined in Persist a store.

You can skip

Skip Wait for hydration with asynchronous storage if you use `localStorage`, and Control hydration for server rendering if you render only in the browser. Symptoms and recoveries are in this chapter's Troubleshooting section, just before the reference.

About persistence

Persist stores an object `{ state, version }` under the key given by `name`. Its `state` field holds `partialize(currentState)`, the part of the store you chose to save. The default storage serializes the object as JSON, which drops functions; actions still exist after loading because the state creator defines them. Values such as a `Map` need encoding.

Persist writes after every `set` and `store.setState` call, even a no-op, once the core store has updated and notified its listeners. It writes nothing at creation except a migrated value. It reads storage during hydration, at creation unless `skipHydration` is `true`.

Without a `storage` option, persist uses `localStorage`. Where storage cannot be obtained, as on a server, persist becomes a pass-through: the store works in memory, `store.persist` is `undefined` despite its type, and each `set` call from the state creator logs a warning.

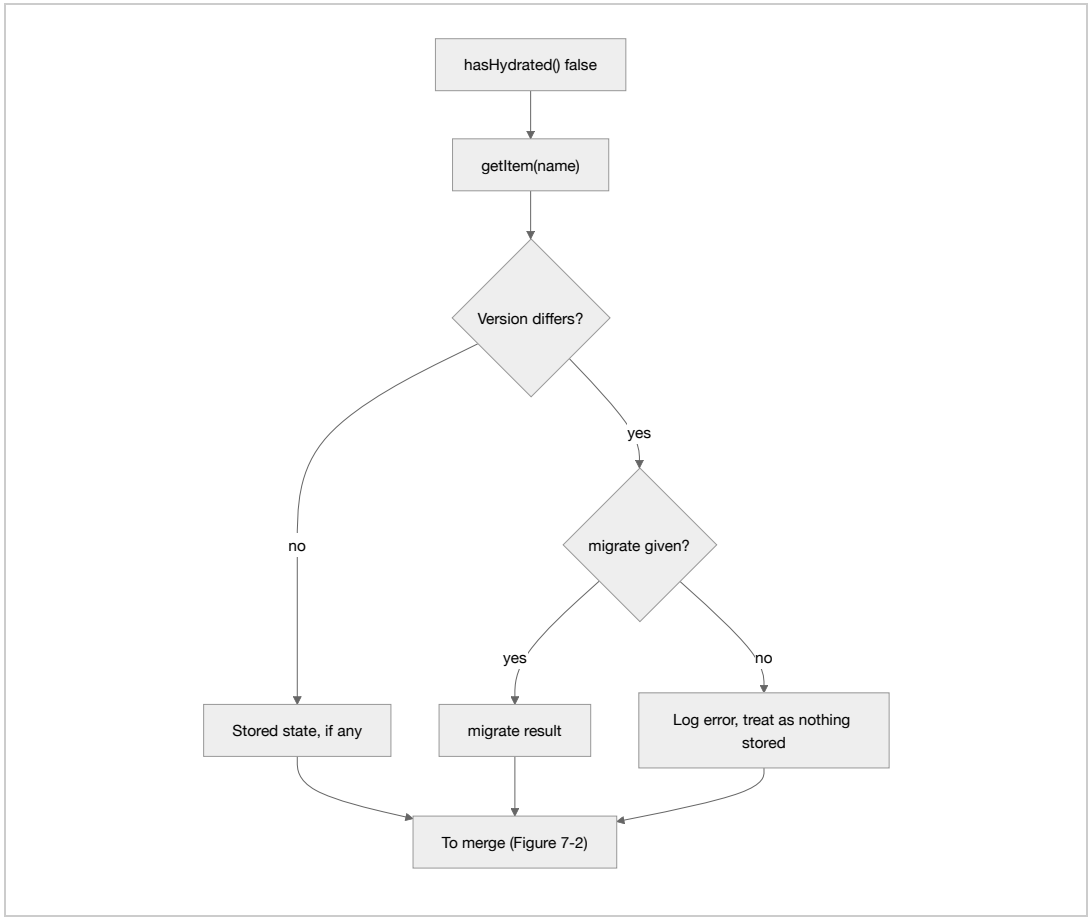
`getInitialState()` returns the state from before hydration, so resets and server rendering use the defaults.

The hydration lifecycle

Hydration runs these steps:

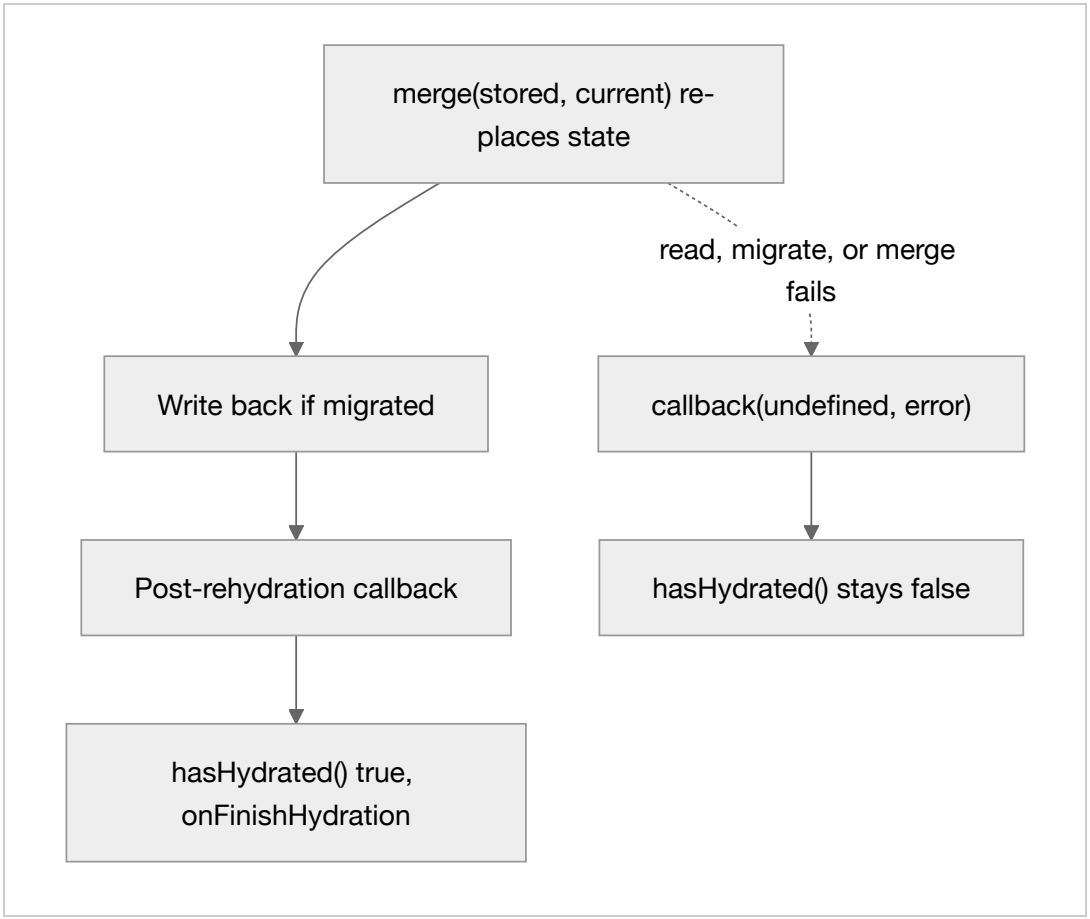
1. Persist marks the store as not hydrated and calls `onHydrate` listeners and `onRehydrateStorage`, which can return a post-rehydration callback.
2. It calls `storage.getItem(name)`.
3. If the stored `version` is a number that differs from the `version` option, it calls `migrate`, or without one logs an error and proceeds as though nothing were stored.
4. It replaces the state with `merge(storedState, currentState)`, which notifies listeners.
5. If a migration ran, it writes the result back.
6. It calls the post-rehydration callback, marks the store hydrated, and calls `onFinishHydration` listeners.

Figure 7-1: Hydration reads storage and uses the stored state as it is, migrates it, or discards it, depending on its version.



Every branch continues to the merge, so a missing `migrate` still produces a hydrated store, built from the defaults.

Figure 7-2: Hydration merges, writes back a migration, and runs the callback before it marks the store hydrated.



Code that waits for `hasHydrated()` or `onFinishHydration` therefore sees the merged stored state, and code that runs earlier sees the defaults.

If reading, parsing, migrating, or merging throws or rejects, persist passes the error to the post-rehydration callback as its second argument and leaves the state unchanged and `hasHydrated()` false. If writing the migrated value fails, the state has already been replaced, although `hasHydrated()` stays false. If the post-rehydration callback or an `onFinishHydration` listener throws, persist calls the callback a second time with that error; when a listener threw, `hasHydrated()` is already `true`.

With synchronous storage, hydration finishes before `create` returns. With asynchronous storage or migration, the store starts with its defaults and hydrates later. A newer `rehydrate()` or `clearStorage()` call cancels a hydration in progress.

Persist a store

Persist the cart's items and coupon but not its checkout status, which describes one submission and is wrong to restore. Because the creator is defined apart from the `persist` call, its type names the persist mutator (How middleware changes the store type). The later listings in this chapter import this file:

```
// persisted-cart.ts
import type { StateCreator } from 'zustand/vanilla'
import type { StateStorage } from 'zustand/middleware'

export type CartItem = { sku: string; name: string; unitPrice: number; quantity:
number }
export type CartState = {
  items: Record<string, CartItem> // keyed by sku
  coupon: string | null
  checkout: { status: 'idle' | 'submitting' }
  addItem: (item: Omit<CartItem, 'quantity'>) => void
}
export type PersistedCart = Pick<CartState, 'items' | 'coupon'>

export const partializeCart = ({ items, coupon }: CartState): PersistedCart =>
  ({ items, coupon })

export const cartCreator: StateCreator<
  CartState,
  [['zustand/persist', unknown]]
> = (set) => ({
  items: {},
  coupon: null,
  checkout: { status: 'idle' },
  addItem: (item) =>
    set(({ items }) => {
      const quantity = (items[item.sku]?.quantity ?? 0) + 1
      return { items: { ...items, [item.sku]: { ...item, quantity } } }
    }),
})

// Synchronous storage backed by a Map, for Node and tests.
export function createMapStorage() {
  const entries = new Map<string, string>()
```

```
getItem: (name) => entries.get(name) ?? null,
    setItem: (name, value) => void entries.set(name, value),
    removeItem: (name) => void entries.delete(name),
  }
  return { ...storage, entries }
}

export const tea = { sku: 'TEA-01', name: 'Sencha tea', unitPrice: 1250 }
```

```
// persist-cart.ts
import { createStore } from 'zustand/vanilla'
import { persist, createJSONStorage } from 'zustand/middleware'
import { cartCreator, createMapStorage, partializeCart, tea, type CartState } from
'./persisted-cart'

const memory = createMapStorage()
const createCartStore = () =>
  createStore<CartState>()(
    persist(cartCreator, {
      name: 'cart',
      storage: createJSONStorage(() => memory),
      partialize: partializeCart,
    }),
  )

createCartStore().getState().addItem(tea)
console.log(memory.entries.get('cart'))
console.log(Object.keys(createCartStore().getState().items)) // as after a reload
```

```
{"state":{"items":{"TEA-01":{"sku":"TEA-01","name":"Sencha
tea","unitPrice":1250,"quantity":1}},"coupon":null},"version":0}
[ 'TEA-01' ]
```

Give each store that shares a storage its own `name`, or they overwrite each other.

Version and migrate persisted state

When a release changes the stored shape, increase `version` and supply `migrate`. Persist calls it for any differing version, including a newer one after a rollback, so treat its input as untrusted. This listing opens the cart of a returning user whose saved cart is at version 0, once with `migrate` and once without:

```
// migrate-cart.ts
import { createStore } from 'zustand/vanilla'
import { persist, createJSONStorage } from 'zustand/middleware'
import {
  cartCreator, createMapStorage, partializeCart, tea,
  type CartItem, type CartState, type PersistedCart,
} from './persisted-cart'

// Version 0 stored items as an array; version 1 keys them by sku.
function migrateCart(persisted: unknown, fromVersion: number): PersistedCart {
  const old = persisted as { items?: CartItem[]; coupon?: string | null } | null
  if (fromVersion !== 0 || !Array.isArray(old?.items)) {
    return { items: {}, coupon: null } // unknown version or shape
  }
  return {
    items: Object.fromEntries(old.items.map((item) => [item.sku, item])),
    coupon: old.coupon ?? null,
  }
}

// Opens the cart of a returning user whose saved cart is at version 0.
function openCart(options: { migrate?: typeof migrateCart }) {
  const memory = createMapStorage()
  const v0 = { state: { items: [{ ...tea, quantity: 2 }], coupon: null }, version: 0
}
  memory.setItem('cart', JSON.stringify(v0))
  const cartStore = createStore<CartState>()(
    persist(cartCreator, {
      name: 'cart',
      storage: createJSONStorage(() => memory),
      partialize: partializeCart,
      version: 1,
      ...options,
    }),
  )
  return { cartStore, memory }
}

const migrated = openCart({ migrate: migrateCart })
console.log(migrated.cartStore.getState().items['TEA-01']?.quantity)
console.log(migrated.memory.entries.get('cart'))

const unmigrated = openCart({})
console.log(unmigrated.cartStore.persist.hasHydrated(),
unmigrated.cartStore.getState().items)
unmigrated.cartStore.getState().addItem(tea)
console.log(unmigrated.memory.entries.get('cart'))
```

```
2
{"state":{"items":{"TEA-01":{"sku":"TEA-01","name":"Sencha
tea","unitPrice":1250,"quantity":2}},"coupon":null},"version":1}
State loaded from storage couldn't be migrated since no migrate function was
provided
true {}
{"state":{"items":{"TEA-01":{"sku":"TEA-01","name":"Sencha
tea","unitPrice":1250,"quantity":1}},"coupon":null},"version":1}
```

With `migrate`, the user keeps the two saved teas, and persist writes the result back at version 1, so each value is migrated once. Without it, persist logs an error, keeps an empty cart, and reports the store as hydrated. The next update then overwrites the old value: the user's two saved teas are gone, and only the tea just added remains.

Merge nested persisted state

The default merge is `{ ...currentState, ...storedState }`, one level deep, so a stored nested object replaces the current one whole. If a release adds `giftWrap: false` to a nested `preferences` object, returning users get no `giftWrap`. Pass a `merge` that merges the nested level too:

```
// merge-preferences.ts
import { createStore } from 'zustand/vanilla'
import { persist, createJSONStorage } from 'zustand/middleware'
import { createMapStorage } from './persisted-cart'

type PreferencesState = {
  preferences: { shipping: 'standard' | 'express'; giftWrap: boolean }
}

const mergePreferences = (persisted: unknown, current: PreferencesState) => {
  const saved = (persisted ?? {}) as Partial<PreferencesState>
  return {
    ...current,
    ...saved,
    preferences: { ...current.preferences, ...saved.preferences },
  }
}

// The current release added giftWrap; an earlier one stored only shipping.
function openPreferences(options: { merge?: typeof mergePreferences }) {
  const memory = createMapStorage()
```

```
memory.setItem('preferences', JSON.stringify(old))
  return createStore<PreferencesState>()(
    persist(() => ({ preferences: { shipping: 'standard', giftWrap: false } }), {
      name: 'preferences',
      storage: createJSONStorage(() => memory),
      ...options,
    }),
  )
}

console.log(openPreferences({}).getState().preferences) // default merge
console.log(openPreferences({ merge: mergePreferences }).getState().preferences)
```

```
{ shipping: 'express' }
{ shipping: 'express', giftWrap: false }
```

`merge` receives `undefined` when nothing usable was stored, and its result becomes the whole state, so keep everything in `current`.

Wait for hydration with asynchronous storage

With storage that returns promises, such as React Native's AsyncStorage, a new store holds its defaults until hydration finishes, which `store.persist.hasHydrated()` and `onFinishHydration` report.

> **WARNING**
> An update made before asynchronous hydration finishes writes state built from the defaults over the stored value, so saved data can be lost. Wait for hydration before updating the store.

This listing gives the cart a storage whose reads take 10 ms. It updates one store before hydration finishes and another after:

```
// early-write.ts
import { createStore } from 'zustand/vanilla'
import { persist, createJSONStorage, type StateStorage } from 'zustand/middleware'
import { cartCreator, partializeCart, tea, type CartState } from './persisted-cart'

// Asynchronous storage backed by a Map. Reads take 10 ms, like AsyncStorage.
const entries = new Map<string, string>()
const slowStorage: StateStorage<Promise<void>> = {
  getItem: (name) =>
    new Promise((resolve) => setTimeout(() => resolve(entries.get(name) ?? null),
```

```
10)),
  setItem: async (name, value) => void entries.set(name, value),
  removeItem: async (name) => void entries.delete(name),
}

const createCartStore = () =>
  createStore<CartState>()(
    persist(cartCreator, {
      name: 'cart',
      storage: createJSONStorage(() => slowStorage),
      partialize: partializeCart,
    }),
  )
type CartStore = ReturnType<typeof createCartStore>

// Resolves when the current hydration succeeds; never resolves if it fails.
const whenHydrated = (store: CartStore) =>
  new Promise<void>((resolve) => {
    if (store.persist.hasHydrated()) return resolve()
    const stop = store.persist.onFinishHydration(() => (stop(), resolve()))
  })

// A returning user saved 3 teas.
const saved = { state: { items: { 'TEA-01': { ...tea, quantity: 3 } }, coupon: null
}, version: 0 }

entries.set('cart', JSON.stringify(saved))
const early = createCartStore()
early.getState().addItem(tea) // before hydration: writes 1 tea over the saved 3
await whenHydrated(early)
console.log('early:', early.getState().items['TEA-01']?.quantity)

entries.set('cart', JSON.stringify(saved))
const gated = createCartStore()
await whenHydrated(gated)
gated.getState().addItem(tea)
console.log('gated:', gated.getState().items['TEA-01']?.quantity)
```

```
early: 1
gated: 4
```

The early update wrote one tea before the delayed read ran, so hydration loaded that write and the three saved teas were lost. When the read runs first instead, hydration replaces the early update. The gated store waited and kept all four.

In React, read the hydration status with a hook whose server snapshot is `false`, so the first client render matches the server. This file also defines the cart's bound hook with the default `localStorage` storage:

```
// use-cart-store.ts
import { useSyncExternalStore } from 'react'
import { create } from 'zustand'
import { persist } from 'zustand/middleware'
import { cartCreator, partializeCart, type CartState } from './persisted-cart'

// Default storage: localStorage in a browser, none on a server.
export const useCartStore = create<CartState>()(
  persist(cartCreator, { name: 'cart', partialize: partializeCart }),
)

const subscribeToHydration = (onChange: () => void) =>
  useCartStore.persist?.onFinishHydration(onChange) ?? (() => {})

export const useHasHydrated = () =>
  useSyncExternalStore(
    subscribeToHydration,
    () => useCartStore.persist?.hasHydrated() ?? true, // no storage to wait for
    () => false, // server rendering and React hydration
  )
```

Render a placeholder and hold back updates while `useHasHydrated()` is `false`. A failed hydration never runs `onFinishHydration`, so handle errors in the post-rehydration callback.

Control hydration for server rendering

Zustand's hooks render `getInitialState()` during React hydration, so they match server markup. With synchronous storage such as `localStorage`, though, the store hydrated when it was created, so a `getState()` call during the first client render sees stored state, and React reports a mismatch. To keep the first client render identical to the server's:

1. Add `skipHydration: true` to the persist options in `use-cart-store.ts`.
2. Near the root of the client tree, call `useCartStore.persist?.rehydrate()` in a `useEffect` with an empty dependency array, so it runs after React hydration. The optional chaining is required because `store.persist` is `undefined` on the server.
3. Gate updates until `rehydrate()` resolves or `useHasHydrated()` is `true`, as the warning in Wait for hydration with asynchronous storage describes.

Step 2 as a component:

```
// cart-hydrator.tsx
import { useEffect } from 'react'
import { useCartStore } from './use-cart-store'

// Render once near the root of the client tree.
export function CartHydrator() {
  useEffect(() => {
    void useCartStore.persist?.rehydrate() // persist is undefined on a server
  }, [])
  return null
}
```

For per-request stores, see Create a store per server request.

Store values JSON cannot represent

JSON stores a `Map` or `Set` as `{}` and a `Date` as a string. Pass `createJSONStorage` a `replacer` that encodes such a value as a tagged object and a `reviver` that rebuilds it from the tag. This store saves products for later as a `Map` from SKU to quantity:

```
// saved-for-later.ts
import { createStore } from 'zustand/vanilla'
import { persist, createJSONStorage } from 'zustand/middleware'
import { createMapStorage } from './persisted-cart'

type SavedState = {
  saved: Map<string, number> // sku -> quantity
  save: (sku: string, quantity: number) => void
}

const memory = createMapStorage()
const storage = createJSONStorage<Pick<SavedState, 'saved'>>(() => memory, {
  replacer: (_key, value) => (value instanceof Map ? { __map: [...value] } : value),
  reviver: (_key, value) => {
    const tagged = value as { __map?: [string, number][] } | null
    return Array.isArray(tagged?.__map) ? new Map(tagged.__map) : value
  },
})

const createSavedStore = () =>
  createStore<SavedState>()(
    persist(
      (set) => ({
        saved: new Map(),
```

```
set(({ saved }) => ({ saved: new Map(saved).set(sku, quantity) })),
      }),
      { name: 'saved', storage, partialize: ({ saved }) => ({ saved }) },
    ),
  )

createSavedStore().getState().save('TEA-01', 2)
console.log(memory.entries.get('saved'))
const reloaded = createSavedStore().getState().saved
console.log(reloaded instanceof Map, reloaded.get('TEA-01'))
```

```
{"state":{"saved":{"__map":[["TEA-01",2]]}},"version":0}
true 2
```

For full control over serialization, implement `PersistStorage` instead.

Clear persisted state

`store.persist.clearStorage()` removes the stored item but leaves the state in memory unchanged, and the next update writes again. To forget a cart at sign-out, call `setState(getInitialState(), true)` first, because the reset is itself a write, and then `clearStorage()`. For tests, see Test persisted stores.

Troubleshooting

Symptom	Cause	Recovery
Saved state is lost on reload	An update ran before asynchronous hydration finished, or `version` changed without `migrate`	Gate updates until the store is hydrated (Wait for hydration with asynchronous storage). Supply `migrate` with every version change (Version and migrate persisted state).
A field added to a nested default is missing for returning users	The default merge replaces a stored nested object whole	Pass a `merge` that merges the nested level (Merge nested persisted state).
React reports a hydration mismatch	The first client render reads stored state the server did not have	Set `skipHydration: true` and call `rehydrate()` in an effect (Control hydration for server rendering).

`[zustand persist middleware] Unable to update item '…', the given storage is currently unavailable.`	No storage exists, as with the default `localStorage` on a server or in Node, or the storage factory threw	In Node and tests, pass a `storage` such as the Map-backed one in Persist a store. On a server the store works in memory, so keep state updates out of server rendering (About persistence).
`store.persist` is `undefined`	Storage is unavailable, so persist adds no `store.persist`	Write `store.persist?.` in code that also runs on a server, and treat a missing `store.persist` as hydrated (Control hydration for server rendering).
Stored state never loads	`name` changed between releases, or another store in the same storage uses the same `name` and overwrites it	Keep `name` fixed across releases, and give each store that shares a storage its own `name` (Persist a store).

Persist reference

Declarations are in `persist.ts` at 5.0.16. Under persist, `set` and `store.setState` return the storage's `setItem` result, a promise you can await for asynchronous storage.

`persist()`

`persist(initializer, options)` returns a state creator that writes as About persistence describes and adds `store.persist`. If a synchronous `setItem` throws, the update has already happened and the error reaches the caller. With asynchronous storage, a failed write rejects the promise that `set` and `setState` return; await or catch it where a lost write matters, because an ignored rejection is unhandled.

`PersistOptions`

Option	Default	Behavior
`name`	Required	Storage key, unique per storage.
`storage`	`localStorage`	`undefined` disables persistence.
`partialize`	Whole state	Returns what to store.
`version`	`0`	A differing stored version triggers `migrate`.
`migrate`	None	`(stored, version)` returns the current shape, or a promise.
`merge`	One-level spread	`(stored, current)` returns the new state.
`onRehydrateStorage`	None	Runs as hydration starts; may return the post-rehydration callback.

`skipHydration`	`false`	Skips hydration at creation.

An option set to `undefined` replaces its default: `merge: undefined` makes every hydration fail, and `partialize: undefined` makes every update throw. Omit an option instead of passing `undefined`.

`store.persist`

Method	Behavior
`getOptions()`, `setOptions(options)`	Read or merge options.
`rehydrate()`	Starts a hydration; awaitable; does not reject on hydration errors.
`hasHydrated()`	`true` after the latest hydration succeeded.
`onHydrate(fn)`, `onFinishHydration(fn)`	Listen for hydration start or success; return an unsubscriber.
`clearStorage()`	Calls `removeItem(name)`; see Clear persisted state.

`createJSONStorage()`

`createJSONStorage(getStorage, { replacer, reviver }?)` adapts a string `StateStorage` into a `PersistStorage`. It calls `getStorage` once and returns `undefined` if that throws. Unparsable stored text fails the hydration.

`StateStorage` and `PersistStorage`

Both have `getItem`, `setItem`, and `removeItem`. A `StateStorage` stores strings, like `localStorage`; a `PersistStorage` stores `StorageValue` objects. A `getItem` that returns a promise makes hydration asynchronous.

`StorageValue`

`{ state: S; version?: number }`, where `state` is the `partialize` output.

CHAPTER 8

Using Middleware

This chapter builds the cart on a typical middleware stack, then covers each bundled middleware except `persist`, which has its own chapter, and `redux`, whose `redux()` entry carries an example.

Roadmap

Read this chapter if

You want DevTools debugging, mutation-style updates, or listeners on one value.

Before you begin

Read How middleware composes, and install the `immer` package.

In this chapter you will

- build the cart with `devtools`, `persist`, and `immer`;
- name updates in the Redux DevTools and write them against an Immer draft;
- run a listener only when one value changes.

You can skip

Write your own middleware, unless you write middleware.

Compose a middleware stack

This module cart uses `immer` for drafts, `persist` for saving, and `devtools` outermost for named actions:

```
// stack-cart.ts
import { create } from 'zustand'
import { createJSONStorage, devtools, persist } from 'zustand/middleware'
import { immer } from 'zustand/middleware/immer'

type CartItem = { sku: string; name: string; unitPrice: number; quantity: number }
type CartState = {
  items: Record<string, CartItem> // keyed by sku; prices in integer cents
  addItem: (item: Omit<CartItem, 'quantity'>) => void
}

const memory = new Map<string, string>() // in a browser, omit `storage`
const storage = createJSONStorage(() => ({
  getItem: (key) => memory.get(key) ?? null,
  setItem: (key, value) => void memory.set(key, value),
  removeItem: (key) => void memory.delete(key),
}))

export const useCartStore = create<CartState>()(
  devtools(
    persist(
      immer((set) => ({
        items: {},
        addItem: (item) =>
          set(
            (draft) => {
              const line = draft.items[item.sku]
              if (line) line.quantity += 1
              else draft.items[item.sku] = { ...item, quantity: 1 }
            },
            undefined,
            { type: 'cart/addItem', sku: item.sku }, // the DevTools action
          ),
      })),
      { name: 'cart', storage },
    ),
    { name: 'cart' },
  ),
)

useCartStore.getState().addItem({ sku: 'TEA-01', name: 'Sencha tea', unitPrice: 1250
})
console.log(memory.get('cart'))
console.log(useCartStore.persist.hasHydrated(), useCartStore.devtools)
```

```
{"state":{"items":{"TEA-01":{"sku":"TEA-01","name":"Sencha
tea","unitPrice":1250,"quantity":1}}},"version":0}
```

```
true undefined
```

Write the stack inline inside `create<CartState>()(…)`, because TypeScript infers each layer's mutator there; that is why `set` accepts a draft and a name, and why `useCartStore.persist` is typed.

Inspect a store with Redux DevTools

The `devtools` middleware sends the initial state and each update to the Redux DevTools extension. Name an update with the third argument to `set` or `setState`: a string, or an object whose `type` names the action and whose other fields appear as the payload, as in the stack above. An unnamed update appears as `anonymous` or an inferred function name.

Without the extension, as in Node, `devtools` is a pass-through and adds no `store.devtools`, which is why the listing printed `undefined`. The type declares the member anyway, so call `store.devtools?.cleanup()` when you discard a store; without optional chaining the call throws a `TypeError`. A jump or reset in the extension replaces the state and notifies listeners without recording an entry. The `devtools()` entry covers the other options.

Write updates as mutations with Immer

The `immer` middleware, from `zustand/middleware/immer`, passes function updaters a draft, as `addItem` uses above. Immer builds the next state with new objects only along the changed paths, so the result follows the Immutability rules. Unchanged lines keep their identity, so components that select them do not re-render.

A draft updater must either change the draft or return a replacement. Doing both throws "An immer producer returned a new value *and* modified its draft" and leaves the state unchanged. An expression-bodied arrow such as `(draft) => (draft.coupon = code)` returns the assigned value, and TypeScript does not catch it, so give draft updaters a block body. `Map` and `Set` fields need Immer's `enableMapSet()` (`immer()`).

Subscribe to part of the state

The `subscribeWithSelector` middleware adds `subscribe(selector, listener, options)`, which calls the listener with the new and previous selected values only when the selected value changes:

```
// subscribe-cart.ts
import { createStore } from 'zustand/vanilla'
import { subscribeWithSelector } from 'zustand/middleware'
import { shallow } from 'zustand/shallow'

type Quantities = { quantities: Record<string, number> }
const cartStore = createStore<Quantities>()(
  subscribeWithSelector((): Quantities => ({ quantities: {} })),
)

cartStore.subscribe(
  (state) => Object.keys(state.quantities),
  (skus, previous) => console.log(previous, '->', skus),
  { equalityFn: shallow },
)

cartStore.setState({ quantities: { 'TEA-01': 1 } })
cartStore.setState({ quantities: { 'TEA-01': 2 } }) // same skus: no output
```

The script prints `[] -> [ 'TEA-01' ]` once. The selector builds a new array on each call, so it passes `shallow` as `equalityFn`; under the default `Object.is`, the listener would run after every update. See `subscribeWithSelector()` for `fireImmediately`.

Infer the state type with combine

`combine(initialState, creator)` merges the initial object with the object `creator` returns and infers the state type from both, so `createStore(combine({ coupon: null as string | null }, (set) => ({ … })))` needs no type argument. Widen initial values as that cast does; without it, `coupon` has type `null`. Because `get()` inside `creator` knows only the initial object's type, declare the type and use `create<T>()(…)` when actions call each other. See `combine()`.

Write your own middleware

If you write middleware. Read How middleware changes the store type first.

A middleware that wraps updates must replace both the `set` it passes to the inner creator and `store.setState`, so that all updates take one path. The wrapper must forward every argument, which carries action names to `devtools`, and return the wrapped result, which carries the promise `persist` returns for asynchronous storage.

The public type of a middleware is written in terms of two mutator lists, named as Zustand's own middleware names them. `Mps` (parent mutators) is the list applied by middleware outside yours; it becomes the `Mis` of the creator you return. `Mcs` (child mutators) is the list the wrapped creator applies; it is that creator's `Mos`. Type a middleware in three steps:

1. Declare a public type, generic in `T`, `Mps`, and `Mcs`, that accepts a `StateCreator<T, Mps, Mcs>` and returns one. Passing both lists through unchanged keeps the types that middleware above and below yours add, such as the draft `set` from `immer`.
2. Implement the middleware against `StateCreator<T, [], []>`. Inside a function generic in `Mps`, TypeScript cannot resolve the mutated types of `set` and `store`, so calls to them do not type-check. The plain store types are callable, and they are all the implementation needs.
3. Cast the implementation to the public type once. TypeScript cannot relate the two creator types without it, and the cast is safe as long as the implementation forwards `set` arguments and reads only `StoreApi` members.

This logging middleware follows those steps and wraps an `immer` layer:

```
// logger.ts
import { create } from 'zustand'
import { immer } from 'zustand/middleware/immer'
import type { StateCreator, StoreMutatorIdentifier } from 'zustand/vanilla'

type Logger = <
  T,
  Mps extends [StoreMutatorIdentifier, unknown][] = [],
  Mcs extends [StoreMutatorIdentifier, unknown][] = [],
>(
  creator: StateCreator<T, Mps, Mcs>,
  label: string,
) => StateCreator<T, Mps, Mcs>

const loggerImpl = <T>(
  creator: StateCreator<T, [], []>,
  label: string,
): StateCreator<T, [], []> => (set, get, store) => {
  const loggedSet: typeof set = (...args) => {
    const result = set(...(args as Parameters<typeof set>)) // forward everything
    console.log(`[${label}]`, JSON.stringify(get()))
    return result
  }
  store.setState = loggedSet // log updates made outside actions too
  return creator(loggedSet, get, store)
}
```

```
export const logger = loggerImpl as Logger

type CartState = {
  quantities: Record<string, number>
  add: (sku: string) => void
}

const useCartStore = create<CartState>()(
  logger(
    immer((set) => ({
      quantities: {},
      add: (sku) =>
        set((draft) => {
          draft.quantities[sku] = (draft.quantities[sku] ?? 0) + 1
        }),
    })),
    'cart',
  ),
)

useCartStore.getState().add('TEA-01')
useCartStore.setState({ quantities: {} })
```

```
[cart] {"quantities":{"TEA-01":1}}
[cart] {"quantities":{}}
```

Because `Mcs` carries `immer`'s mutator through `logger`, `useCartStore.setState` also accepts a draft updater; a `Logger` type that returned `StateCreator<T, Mps, []>` would drop it. The second line comes from `setState`, which `logger` replaced.

A middleware that adds a store member must also declare a mutator, so that the store type gains the member. Augment `StoreMutators` in `zustand/vanilla` with an identifier that maps the store type `S` to the changed type; prefix the identifier with your package name, because every declaration shares one interface. Then append the mutator to `Mps` for the creator you accept, so the inner creator's `store` has the member, and prepend it to `Mcs` for the creator you return, so `create` applies it. The second element of each pair is `A` in the augmentation, a type argument the mutator can use; pass `never` when it needs none. This middleware adds `store.reset()`:

```
// resettable.ts
import { createStore } from 'zustand/vanilla'
import type { StateCreator, StoreApi, StoreMutatorIdentifier } from
'zustand/vanilla'
```

```
interface StoreMutators<S, A> {
    'example/resettable': Omit<S, 'reset'> & { reset: () => void }
  }
}

type Resettable = <
  T,
  Mps extends [StoreMutatorIdentifier, unknown][] = [],
  Mcs extends [StoreMutatorIdentifier, unknown][] = [],
>(
  creator: StateCreator<T, [...Mps, ['example/resettable', never]], Mcs>,
) => StateCreator<T, Mps, [['example/resettable', never], ...Mcs]>

const resettableImpl = <T>(
  creator: StateCreator<T, [], []>,
): StateCreator<T, [], []> => (set, get, store) => {
  const initial = creator(set, get, store)
  ;(store as StoreApi<T> & { reset: () => void }).reset = () => set(initial, true)
  return initial
}

export const resettable = resettableImpl as Resettable

const counter = createStore<{ count: number }>()(resettable(() => ({ count: 0 })))
counter.setState({ count: 5 })
counter.reset() // typed through the mutator
console.log(counter.getState().count) // 0
```

The implementation still sees the plain store type, so it casts `store` once to assign the new member.

Troubleshooting

Symptom	Cause	Recovery
DevTools shows `anonymous` action names	No name was passed, or a middleware dropped it	Pass a name to `set`; forward all arguments
"An immer producer returned a new value *and* modified its draft"	A draft updater mutated and returned	Use a block body
Middleware "not assignable to parameter of type 'StateCreator<CartState, [], []>'"	`create` was not curried	Write `create<CartState>()(…)`
"Parameter 'draft' implicitly has an 'any' type"	The stack was built outside `create`	Write it inline inside `create<CartState>()(…)`

CHAPTER 9

Testing Stores

A store is a plain object that a test can create, drive, and inspect, so most store tests need no React. This chapter tests the cart from Chapters 5 to 7 with Node's built-in test runner.

Roadmap

Read this chapter if

You write unit tests for stores, for components that read stores, or for persisted state.

Before you begin

- The tests import `cart.ts` from Define a store with state and actions. The component test also imports `cart-context.tsx` and `scoped-components.tsx` from Scope a store to a component tree.
- Run test files with `npx tsx --test`. To type-check them, install `@types/node` and add `"node"` to `compilerOptions.types`.

In this chapter you will

1. Test checkout against a fake backend.
2. Reset module stores between tests.
3. Test components and persisted stores.

You can skip

Test persisted stores if you do not use `persist`.

Test store logic without React

Create a new store in each test with `createCartStore()`. Each call returns an independent store, so no test sees another test's state. Pass a fake `OrdersApi` that rejects the order:

```
// checkout.test.ts
import { test } from 'node:test'
import assert from 'node:assert/strict'
import { createCartStore, MUG, OrderRejectedError, type OrdersApi } from './cart'

const rejectingApi: OrdersApi = {
  submit: async () => {
    throw new OrderRejectedError('MUG-02 is out of stock')
  },
}

test('a rejected order keeps the cart so the user can retry', async () => {
  const store = createCartStore()
  const statuses: string[] = []
  store.subscribe((state, previous) => {
    if (state.checkout !== previous.checkout) statuses.push(state.checkout.status)
  })
  store.getState().addItem(MUG)
  const itemsBefore = store.getState().items

  await store.getState().submitOrder(rejectingApi)

  assert.deepEqual(statuses, ['submitting', 'failed'])
  assert.equal(store.getState().checkout.error, 'MUG-02 is out of stock')
  // Same reference: the failure left the cart untouched, so a retry is safe.
  assert.equal(store.getState().items, itemsBefore)
})
```

`getState()` returns the latest state synchronously, so assertions after `await submitOrder(…)` see the final state. The `submitting` status exists only while the request is pending, so a listener attached beforehand records it (How listeners are notified).

Reset module stores between tests

A module store, such as `useCartStore`, lives as long as the process that imported it, so items one test adds are still there when the next test starts. Node's test runner gives each file its own process, so the leak stays within one file.

Restore the initial state before each test with `setState(getInitialState(), true)`, the reset described in Reset a store. When tests touch several module stores, record a reset function for each in a small registry:

```
// store-resets.ts
import type { StoreApi } from 'zustand'

const resets = new Set<() => void>()

export function resetBetweenTests<S>(store: StoreApi<S>): StoreApi<S> {
  resets.add(() => store.setState(store.getInitialState(), true))
  return store
}

export function resetAllStores(): void {
  for (const reset of resets) reset()
}
```

A test file calls `resetBetweenTests(useCartStore)` and `beforeEach(resetAllStores)`. A bound hook carries the store API, so `resetBetweenTests` accepts `useCartStore` directly.

A reset does not remove listeners. A subscription made in one test keeps running in later tests, so unsubscribe when the test ends, for example with `t.after(unsubscribe)`. For Jest and Vitest, Zustand's testing guide shows a module mock of `zustand` that resets every store automatically.

Test components with an injected store

Render a component that calls `useCartStoreContext` under a `CartStoreProvider` with test props, such as `initialItems`. Each render mounts a new provider with a new store, so these tests need no reset. `renderToString` from `react-dom/server` renders the store's initial state (Server rendering and the initial state), which for a scoped store is the state built from the provider's props:

```
// components.test.tsx
import { test } from 'node:test'
import assert from 'node:assert/strict'
import { renderToString } from 'react-dom/server'
import { MUG, TEA } from './cart'
import { CartStoreProvider } from './cart-context'
import { CartBadge } from './scoped-components'
```

```
const items = {
    [TEA.sku]: { ...TEA, quantity: 2 },
    [MUG.sku]: { ...MUG, quantity: 1 },
  }
  const html = renderToString(
    <CartStoreProvider initialItems={items}>
      <CartBadge />
    </CartStoreProvider>,
  )
  assert.equal(html, '<span aria-label="Items in cart">3</span>')
})

test('the badge outside a provider throws', () => {
  assert.throws(() => renderToString(<CartBadge />), /inside <CartStoreProvider>/)
})
```

Running `npx tsx --test components.test.tsx` prints:

```
✔ the badge counts every unit in the provided cart (2.678959ms)
✔ the badge outside a provider throws (0.231333ms)
ℹ tests 2
ℹ suites 0
ℹ pass 2
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 127.896583
```

The durations differ from run to run. Tests that click and re-render need a DOM environment, such as jsdom with Testing Library, which this book does not cover.

Test persisted stores

This test builds the persisted cart from Persist a store, saved as `persisted-cart-model.ts`, on a Map-backed storage, which it seeds with the JSON that `persist` writes and empties before each test:

```
// persisted-cart.test.ts
import { beforeEach, test } from 'node:test'
import assert from 'node:assert/strict'
import { createStore } from 'zustand/vanilla'
import { createJSONStorage, persist } from 'zustand/middleware'
import { cartCreator, createMapStorage, partializeCart, tea, type CartState } from
```

```
const storage = createMapStorage()
beforeEach(() => storage.entries.clear())

const createCart = () =>
  createStore<CartState>()(
    persist(cartCreator, {
      name: 'cart',
      storage: createJSONStorage(() => storage),
      partialize: partializeCart,
    }),
  )

test('hydrates from stored JSON and writes only persisted fields', () => {
  const state = { items: { 'TEA-01': { ...tea, quantity: 3 } }, coupon: 'WELCOME10'
}
  storage.entries.set('cart', JSON.stringify({ state, version: 0 }))

  const store = createCart()
  assert.equal(store.getState().items['TEA-01']?.quantity, 3)

  store.getState().addItem(tea)
  assert.deepEqual(JSON.parse(storage.entries.get('cart') ?? 'null').state, {
    items: { 'TEA-01': { ...tea, quantity: 4 } },
    coupon: 'WELCOME10',
  })
})
```

Synchronous storage hydrates while the store is created, so the test can assert at once. With asynchronous storage, `await store.persist.rehydrate()` first (Wait for hydration with asynchronous storage). Create the store inside each test, after seeding storage, because a store created at module scope hydrates once and never reads what a later test seeds.

Troubleshooting

Symptom	Cause	Recovery
State from one test appears in the next	A module store lives for the whole test file	Reset it in `beforeEach` (Reset module stores between tests), or create a store per test
A listener from an earlier test still runs	Resetting state does not remove listeners	Unsubscribe when the test ends, for example with `t.after(unsubscribe)`
A persisted store ignores the storage a test seeded	The store was created at module scope and hydrated before the seed	Create the store inside the test after seeding, or call `await store.persist.rehydrate()`

A persisted store shows default state with asynchronous storage	The test asserted before hydration finished	Call `await store.persist.rehydrate()` before asserting

CHAPTER 10

Zustand Reference

This chapter specifies the public contract of Zustand 5.0.16 for lookup, with one entry for each function and type you call or configure. The `persist` middleware is specified in Persist reference in Chapter 7. Core declarations follow the Zustand source. Middleware signatures omit the mutator type parameters (How middleware changes the store type).

Roadmap

Read this chapter if

You need the exact behavior of a Zustand function or type.

Before you begin

Read Stores and State Updates. The entries use its terms and link to its sections instead of repeating them.

In this chapter you will

- find which entry point exports a name, in Entry points, the book's interface summary;
- look up the core store, the React bindings, the equality helpers, and the middleware.

You can skip

Everything you are not looking up.

Entry points

The packages in the last column are optional peer dependencies, so install one only if you import an entry point that needs it. Requirements gives their minimum versions. Every export is named; there are no default exports.

Entry point	Exports	Needs
`zustand`	Everything in `zustand/vanilla` and `zustand/react`	`react`
`zustand/vanilla`	`createStore`, `StoreApi`, `StateCreator`, `ExtractState`; `Mutate`, `StoreMutators`, and `StoreMutatorIdentifier` (store types)	Nothing
`zustand/react`	`create`, `UseBoundStore`, `useStore`	`react`
`zustand/middleware`	`devtools`, `DevtoolsOptions`, `NamedSet`; `subscribeWithSelector`; `combine`; `redux`; `persist`, `PersistOptions`, `createJSONStorage`, `StateStorage`, `PersistStorage`, `StorageValue`; `unstable_ssrSafe`	Nothing
`zustand/middleware/immer`	`immer`	`immer`
`zustand/shallow`	`shallow`, `useShallow`	`react`, even if you use only `shallow`
`zustand/vanilla/shallow`	`shallow`	Nothing
`zustand/react/shallow`	`useShallow`	`react`
`zustand/traditional`	`createWithEqualityFn`, `UseBoundStoreWithEqualityFn`, `useStoreWithEqualityFn`	`react`, `use-sync-external-store`

The entry points that need nothing import nothing from React and run in any JavaScript runtime.

Shared rules

These rules hold for every entry in this chapter.

- Everything is synchronous except `persist` hydration and writes when the storage or `migrate` returns a promise.

- Zustand keeps your values by reference and never clones or freezes them, so you must create a new object for each level you change (Immutability rules).
- Selectors, equality functions, and listeners can run on every update and every render. They must be pure, because React can call a selector several times per render, and they should be cheap, because their cost adds to every update.
- The core store throws no errors of its own and catches none of yours (Store invariants).
- The React APIs are hooks built on `useSyncExternalStore`, so they need React 18 or later.

`createStore()`

Creates a vanilla store from a state creator and returns the store API.

```
type CreateStore = {
  <T, Mos extends [StoreMutatorIdentifier, unknown][] = []>(
    initializer: StateCreator<T, [], Mos>,
  ): Mutate<StoreApi<T>, Mos>

  <T>(): <Mos extends [StoreMutatorIdentifier, unknown][] = []>(
    initializer: StateCreator<T, [], Mos>,
  ) => Mutate<StoreApi<T>, Mos>
}
```

Returns

A `StoreApi<T>` with any members that middleware adds. The curried form, `createStore<T>()(initializer)`, lets you write `T` while TypeScript infers the middleware's mutators (Why `create` is curried in TypeScript).

Behavior

Calls `initializer(setState, getState, api)` once, synchronously, and stores its return value as both the current state and the initial state. A store lives as long as something references it.

See also

How a store is created and Define a store with state and actions.

`StoreApi`

The four methods that every store exposes and that a bound hook carries.

```
// Simplified; the full declaration is in src/vanilla.ts.
export interface StoreApi<T> {
  setState: {
    (partial: T | Partial<T> | ((state: T) => T | Partial<T>), replace?: false):
void
    (state: T | ((state: T) => T), replace: true): void
  }
  getState: () => T
  getInitialState: () => T
  subscribe: (listener: (state: T, prevState: T) => void) => () => void
}
```

Members

Member	Behavior
`getState()`	Returns the current state.
`setState(partial)`	Merges a partial state, or an updater's result, one level deep (How setState computes the next state).
`setState(state, true)`	Replaces the state with a complete T, removing omitted fields, including actions.
`subscribe(listener)`	Calls `listener(state, prevState)` after each change and returns an unsubscribe function (How listeners are notified).
`getInitialState()`	Returns the state the state creator returned, which React uses as the server snapshot.

The full declaration expresses the two `setState` forms with an internal helper type. The methods do not use `this`, so you can destructure them. Middleware can replace `setState` and `subscribe` and add members.

`StateCreator`

The type of the function that a store or a middleware calls once to produce state.

```
export type StateCreator<
  T,
  Mis extends [StoreMutatorIdentifier, unknown][] = [],
  Mos extends [StoreMutatorIdentifier, unknown][] = [],
  U = T,
> = ((
  setState: Get<Mutate<StoreApi<T>, Mis>, 'setState', never>,
  getState: Get<Mutate<StoreApi<T>, Mis>, 'getState', never>,
  store: Mutate<StoreApi<T>, Mis>,
) => U) & { $$storeMutators?: Mos }
```

Type parameters

Name	Meaning
`T`	The full state type of the store.
`Mis`	Mutators from middleware outside this creator, which type its arguments.
`Mos`	Mutators this creator applies, which determine the store's type.
`U`	The returned type. A slice's `U` is narrower than `T`.

The function must return synchronously, because a returned promise becomes the state itself.

See also

Split a large store into slices and Write your own middleware.

`create()`

Creates a store and returns a bound hook: a React hook that reads the store, with the store API attached.

```
export type UseBoundStore<S extends ReadonlyStoreApi<unknown>> = {
  (): ExtractState<S>
  <U>(selector: (state: ExtractState<S>) => U): U
} & S
```

Parameters

The same as `createStore()`, in the same direct and curried forms.

Behavior

Calls `createStore(initializer)` and copies the API's members onto the hook, so `useCartStore.getState()` works outside components. A hook call is `useStore()` with the store applied. The hook takes no equality function, so use `useShallow()` or `createWithEqualityFn()` instead. A module-scope hook is one store shared by every component, test, and server request (Module stores and scoped stores).

See also

How a component subscribes to a store and Read a value in a component.

`useStore()`

Reads a store in a React component and re-renders the component when the selected value changes.

```
export function useStore<S extends ReadonlyStoreApi<unknown>>(
  api: S,
): ExtractState<S>

export function useStore<S extends ReadonlyStoreApi<unknown>, U>(
  api: S,
  selector: (state: ExtractState<S>) => U,
): U

export type ExtractState<S> = S extends { getState: () => infer T } ? T : never
```

`ReadonlyStoreApi<T>` is `StoreApi<T>` without `setState`, so `api` can be a vanilla store or a bound hook. The default `selector` returns the whole state.

Behavior

Subscribes through `useSyncExternalStore`, using `getInitialState()` for the server snapshot, and re-renders when the selected value differs under `Object.is` (When a component re-renders).

Errors

A selector that returns a new object, array, or function on every call makes React warn that the snapshot should be cached, and can end in "Maximum update depth exceeded" (Selectors must return stable values).

See also

Use a vanilla store in React and Scope a store to a component tree.

`useShallow()`

Wraps a selector so that it keeps returning its previous result while each new result is shallowly equal to it.

```
export function useShallow<S, U>(selector: (state: S) => U): (state: S) => U
```

Behavior

Each call of the returned selector runs `selector` and compares the result with the previous one using `shallow()`. If they are equal, it returns the previous result, so the reference does not change. It keeps the previous result in a React ref, so it is a hook: call it inline, as in `useCartStore(useShallow((s) => …))`.

See also

Shallow comparison with `useShallow` and Select several values.

`shallow()`

Compares two values one level deep and returns the result of the first rule that applies.

```
export function shallow<T>(valueA: T, valueB: T): boolean
```

Rule	Result
`Object.is(valueA, valueB)`	`true`
Either value is `null` or not an object	`false`

The prototypes differ	`false`
Both are iterable and have an `entries` member, such as arrays, `Map`, and `Set`	`true` if both have the same keys with `Object.is`-equal values
Both are iterables without `entries`	`true` if their values are `Object.is`-equal position by position
Otherwise	`true` if both have the same own enumerable keys with `Object.is`-equal values

Nested objects compare by reference. Shallow comparison with `useShallow` shows example pairs and their results.

`createWithEqualityFn()`

Creates a store and returns a bound hook whose calls accept an equality function, with an optional default for the store.

```
createWithEqualityFn(initializer, defaultEqualityFn?: <U>(a: U, b: U) => boolean)
useBoundStore(selector, equalityFn?: (a: U, b: U) => boolean)
```

Behavior

Takes the same forms as `create()` and returns a `UseBoundStoreWithEqualityFn`. Each hook call is `useStoreWithEqualityFn()` with the per-call equality function, or else `defaultEqualityFn`, or else `Object.is`.

Errors

Importing `zustand/traditional` fails if `use-sync-external-store` is not installed.

See also

Custom equality with `zustand/traditional` and Changes from Zustand 4.

`useStoreWithEqualityFn()`

Reads a store in a React component and re-renders only when an equality function reports that the selected value changed.

```
export function useStoreWithEqualityFn<S extends ReadonlyStoreApi<unknown>, U>(
  api: S,
  selector: (state: ExtractState<S>) => U,
  equalityFn?: (a: U, b: U) => boolean,
): U
```

A second overload takes only `api` and returns the whole state. The default `equalityFn` is `Object.is`.

Behavior

Subscribes with `useSyncExternalStoreWithSelector` from `use-sync-external-store`. While `equalityFn(previous, next)` returns `true`, the hook returns the previous selected value and the component does not re-render.

`devtools()`

Connects a store to the Redux DevTools browser extension, which records each update as an action.

```
devtools(initializer: StateCreator<T>, devtoolsOptions?: DevtoolsOptions)

export interface DevtoolsOptions extends Config {
  name?: string
  enabled?: boolean
  anonymousActionType?: string
  store?: string
}
```

Parameters

Option	Default	Description
`name`	none	The instance name the extension shows.
`enabled`	`true` outside production builds	`false` makes `devtools` a pass-through. The ES module build stays enabled if `import.meta.env` is undefined, so pass `enabled` when your bundler does not define it.
`anonymousActionType`	inferred from the stack	The action type for an update that names no action.
`store`	none	Shares one connection among stores with the same `name`.

Other fields are the extension's connection options.

Behavior

When the extension is present and `enabled` holds, `devtools` replaces `setState` with one that takes a third argument, `action` (a string or an object with a `type`), and adds `store.devtools.cleanup()`. Otherwise it is a pass-through that adds no `store.devtools` (Inspect a store with Redux DevTools).

Apply `devtools` outermost, because it records only updates that pass through its wrapper; a layer outside it, such as `persist` during hydration, updates the store unrecorded (How middleware composes). `devtools` throws nothing of its own.

`immer()`

Lets function updaters produce the next state by mutating an Immer draft. Import it from `zustand/middleware/immer`, which requires the `immer` package.

```
immer(initializer: StateCreator<T>): StateCreator<T>
```

Behavior

Replaces `setState` so that a function updater runs through Immer's `produce` with a draft of the current state; mutate the draft and return nothing, or return a complete replacement. Other arguments pass through unchanged. An updater that changes nothing returns the same state object, so the store does not notify.

Errors

Errors from Immer propagate out of `setState`, and the state is unchanged. An updater that both modifies the draft and returns a value throws (Write updates as mutations with Immer), and so does reading or modifying a `Map` or `Set` field of the draft before you call Immer's `enableMapSet()`.

See also

Write updates as mutations with Immer.

`subscribeWithSelector()`

Extends `subscribe` so that a listener can watch one selected value and run only when it changes.

```
subscribeWithSelector(initializer: StateCreator<T>): StateCreator<T>

// Added overload of store.subscribe:
subscribe<U>(
  selector: (state: T) => U,
  listener: (selectedState: U, previousSelectedState: U) => void,
  options?: {
    equalityFn?: (a: U, b: U) => boolean
    fireImmediately?: boolean
  },
): () => void
```

Behavior

The selector form calls the listener only when `equalityFn(lastDelivered, next)` returns `false`; the default is `Object.is`, and `shallow` suits objects and arrays. With `fireImmediately: true` it calls `listener(current, current)` before `subscribe` returns. The one-argument form is the original `subscribe`, so React hooks behave as before.

See also

Subscribe to part of the state.

`combine()`

Builds a state creator from an initial data object and a creator for the remaining fields, and infers the state type from both.

```
combine(initialState: T, create: StateCreator<T, [], [], U>): StateCreator</* T
merged with U */>
```

Returns

The state creator for `T` merged with `U`, where a field of `U` wins over a field of `T` with the same name.

Behavior

Calls `create` with the store's arguments, typed by `T` only, and returns `Object.assign({}, initialState, result)`. Nested objects in `initialState` are shared by reference, including between stores created from one `combine` call, so they must not be mutated. The state type comes from the initial values, so widen any field whose type is wider than its initial value (Infer the state type with combine).

See also

Infer the state type with combine.

`redux()`

Builds a state creator that updates the state with a reducer, and adds `dispatch` to the state and to the store API.

```
redux(reducer: (state: T, action: A) => T, initialState: T): StateCreator<T & {
dispatch }>
```

Behavior

`dispatch(action)`, on the store and on the state, calls `set((state) => reducer(state, action), false, action)` and returns `action`. The result is merged, so `dispatch` survives a reducer that returns only data fields. Under `devtools(redux(…))`, each action is recorded under its `type`. A reducer exception propagates out of `dispatch` and leaves the state unchanged.

Example

```
import { createStore } from 'zustand/vanilla'
import { redux } from 'zustand/middleware'

type Counts = { counts: Record<string, number> }
type CartAction = { type: 'cart/add'; sku: string } | { type: 'cart/clear' }

const initialCounts: Counts = { counts: {} }

function cartReducer(state: Counts, action: CartAction): Counts {
  switch (action.type) {
```

```
return { counts: { ...state.counts, [action.sku]: (state.counts[action.sku] ?? 0) +
1 } }
    case 'cart/clear':
      return initialCounts
  }
}

const cartStore = createStore(redux(cartReducer, initialCounts))
cartStore.dispatch({ type: 'cart/add', sku: 'TEA-01' })
cartStore.dispatch({ type: 'cart/clear' })
console.log(Object.keys(cartStore.getState()))
```

```
[ 'dispatch', 'counts' ]
```

`dispatch` survives `cart/clear` because the reducer's result is merged.

APPENDIX A

Glossary

Each entry defines a term as this book uses it and links to the section that explains it.

action: A function-valued field of the state that updates the store through `set`, such as `addItem`. See Actions and where logic lives.

bound hook: The React hook `create` returns, such as `useCartStore`, which also carries the store API methods for code outside React. See How a component subscribes to a store. **Compare** vanilla store.

equality function: A function that receives the previous and next selected values and returns `true` when they count as equal, so the component does not re-render or the listener is not called. The hooks from `zustand/traditional` accept one per call or per store, and `subscribeWithSelector` accepts one as `equalityFn`. `create` and `useStore` accept none and compare with `Object.is`. See Custom equality with `zustand/traditional`. **Compare** shallow equality.

hydration: The process in which `persist` reads stored state, migrates it if needed, and merges it into the current state. With asynchronous storage it finishes after creation. See The hydration lifecycle.

initial state: The value the state creator returned at creation, which `getInitialState()` returns for the life of the store. See Reset a store.

listener: A function passed to `subscribe` that receives `(state, previousState)`. The store calls it synchronously inside `setState`. See How listeners are notified.

merge: (1) How `setState` combines a partial state with the current state, one level deep (How setState computes the next state). (2) The `persist` option that combines stored and current state during hydration (Merge nested persisted state).

middleware: A function that takes a state creator and returns one, and can wrap `set` or extend the store API. See What middleware is.

migration: The `migrate` call that `persist` makes during hydration when the stored version is a number that differs from the `version` option. It converts persisted state to the current shape, and `persist` writes the result back, so each stored value is migrated once. Without `migrate`, `persist` logs an error and keeps the defaults. See Version and migrate persisted state.

module store: A store created at the top level of a module, so every component, test, and server request in the process shares it. See Module stores and scoped stores. **Compare** scoped store.

mutator: A type-level change that a middleware makes to the store type, registered in the `StoreMutators` interface and written as a pair such as `['zustand/persist', unknown]`. A middleware stack produces an ordered list of mutators, which `create` applies to `StoreApi<T>`; slices under middleware and custom middleware pass these lists through the type parameters of `StateCreator`. A mutator changes only types; the middleware itself makes the runtime change. See How middleware changes the store type. **Compare** middleware.

partial state: An object with only some of the state's fields, passed to `set` or `setState` or returned by an updater. Without the replace flag the store merges it one level deep into a new state object, so the fields it omits, including actions, keep their values. See How setState computes the next state. **Compare** replace flag.

`partialize`: The `persist` option that chooses what to store: a function from the current state to the persisted state, called on every write. By default `persist` stores the whole state. Use it to leave out fields that are wrong to restore, such as a request status. See Persist a store and `PersistOptions`.

persisted state: The value `persist` stores under its `name`, as the `state` field of a `StorageValue`. It is the `partialize` output at the last write, and hydration passes it to `migrate` and `merge`. Because it can come from an older or newer release, treat it as untrusted input. See About persistence. **Compare** initial state.

replace flag: The second argument of `set` and `setState`. With `true`, the given value becomes the whole state, so omitted fields, including actions, disappear. See How setState computes the next state.

scoped store: A store that a provider component creates and shares with its descendants through React context, one instance per mounted provider. See Scope a store to a component tree. **Compare** module store.

selected value: What a selector returns. A component re-renders only when it changes under `Object.is`. Zustand's type parameters call this value a slice; this book does not. See When a component re-renders.

selector: A pure function from the state to a selected value, which runs on render and after every update. See Selectors must return stable values.

shallow equality: The comparison `shallow` performs: `Object.is` on each top-level entry, so nested objects match only by reference. See Shallow comparison with `useShallow`.

slice: In this book, a state creator that builds part of a store's state and is combined with other slices in one store. See Split a large store into slices. **Compare** selected value.

state: The single value a store holds, usually an object of data and actions. Changes are detected by reference, so each update must create new objects. See Immutability rules.

state creator: The function `(set, get, store) => initialState` passed to `create`, `createStore`, or a middleware, and called once at creation. See How a store is created.

storage: The object `persist` reads and writes, given as its `storage` option. A `PersistStorage` stores `StorageValue` objects; a `StateStorage` stores strings, as `localStorage` does, and `createJSONStorage` adapts one into a `PersistStorage`. A `getItem` that returns a promise makes hydration asynchronous. See `StateStorage` and `PersistStorage`.

store: A container for one current state value and its listeners, independent of React and reached through the store API. See What a store is and `StoreApi`.

store API: The object with `getState`, `setState`, `subscribe`, and `getInitialState` through which code reaches a store. `createStore` returns it, `create` attaches its methods to the bound hook, and a state creator receives it as its third argument. Middleware can replace its methods or add members such as `store.persist`. See `StoreApi`.

subscription: A listener registered with `subscribe`, which stays registered until you call the unsubscribe function that `subscribe` returns. Until then the store keeps the listener, and everything the listener closes over, alive. A component subscribes through `useStore`, and React manages that subscription. See How listeners are notified. **See also** listener.

transient update: A state change that code handles in a `subscribe` listener instead of through a render, such as updating a page title or a log. Frequent changes handled this way cost no React render. See React to changes without rendering.

vanilla store: The store API object `createStore` returns, with no hook. Components read it with `useStore`. See `createStore()`. **Compare** bound hook.

APPENDIX B

Errors and Symptoms

Find the message or behavior you observed (`…` marks varying text) and follow the link to its cause and recovery. React's getSnapshot warning prints only in development; in production React throws `Maximum update depth exceeded` as minified error #185. The devtools warning prints wherever devtools runs outside production mode.

Message or symptom	Likely cause	See
`Maximum update depth exceeded`, `Minified React error #185`, or `The result of getSnapshot should be cached…` (React)	A selector returns a new object, array, or function each call	Stable selectors
`[zustand persist middleware] Unable to update item '…', the given storage is currently unavailable.`	Storage is missing, as on a server, or its factory threw	About persistence
`State loaded from storage couldn't be migrated since no migrate function was provided`	The stored version differs from `version`, with no `migrate`	Migration
`[zustand devtools middleware] "__setState" action type is reserved…`	A `redux` store dispatched `__setState` under `devtools`	`devtools()`
A component does not re-render after `set`	The update mutated state in place	Immutability rules
A component re-renders on every change	No selector, or the selector returns the whole state	Re-renders
React hydration mismatch with a persisted store	The first client render reads stored state the server lacked	Hydration and SSR
Saved state replaced by defaults on reload	An update ran before asynchronous hydration finished	Async hydration
`store.persist` or `store.devtools` is `undefined`	Storage is unavailable, or the extension is absent or disabled, as in production	About persistence, Redux DevTools

APPENDIX C

Compatibility and Migration

This appendix gives the requirements of Zustand 5.0.16 and the changes from Zustand 4 that require code edits.

Requirements

Zustand's four peer dependencies are all optional, so a package manager does not install them. Install each one that your code needs, because an entry point whose peer is missing fails to load. Entry points shows which entry point needs which peer.

Package	Minimum version
`react`	18.0.0
`@types/react`	18.0.0
`use-sync-external-store`	1.2.0
`immer`	9.0.6

Zustand's types require TypeScript 4.5 or later.

Each entry point ships an ES module build and a CommonJS build, and the package's `exports` map selects one for `import` or `require` without configuration. The map also has a `react-native` condition, which this book does not cover. `zustand/vanilla` and `zustand/middleware` do not import React, so stores built from them run in any JavaScript runtime.

Changes from Zustand 4

Each change below requires edits to code written for Zustand 4. Zustand's migration guide covers the rest.

- There are no default exports. Replace `import create from 'zustand'` with `import { create } from 'zustand'`.
- The bound hook and `useStore` accept no equality function. TypeScript rejects one, but at run time Zustand ignores it without a warning. Wrap the selector with `useShallow`, or switch to `zustand/traditional`.
- Selectors must return stable values. Zustand 4 cached selected values, so a selector that built a new object on each call still worked. In Zustand 5 it can fail with "Maximum update depth exceeded" (Selectors must return stable values).
- Under TypeScript, `setState(next, true)` requires `next` to be a complete state, including actions (How setState computes the next state).
- `persist` no longer writes to storage when the store is created, only at the first update (About persistence). If your code expects the initial state in storage, call `store.setState(store.getState())` once hydration has finished, for example in a `store.persist.onFinishHydration` listener or after `await store.persist.rehydrate()`. With synchronous storage, calling it right after creation is safe. With asynchronous storage, an earlier call writes the defaults over the saved state before hydration reads it (Wait for hydration with asynchronous storage).
- `use-sync-external-store` is now an optional peer dependency, so a project that imports `zustand/traditional` must add it. The React floor rose from 16.8 to 18 (Requirements).

Experimental and unstable APIs

The book documents no export with the `unstable_` prefix, which Zustand uses to mark experimental APIs. In Zustand 5.0.16 the only one is `unstable_ssrSafe` in `zustand/middleware`. If you use it, pin the exact Zustand version, because an upgrade can change or remove it.